

长程实践：超越单次能力的结构下 限

Long-Horizon Practice: Structural Lower Bounds Beyond Single-Step Capability

曾庆明 (Sammy Zeng)

魏道 (Dao Wei)

2026 年

¹联系邮箱: jop.sammy@163.com ORCID: [0009-0007-4382-0040](https://orcid.org/0009-0007-4382-0040)

²本文为杭州市上城区钱江新城中央商务产业社区专项支持项目“长程实践理论”的研究成果，社区在调研对接、基础压力测试样例、真实业务用例方面提供了关键性支持（非经费资助）

Abstract

现有对智能体（Agent）能力的讨论，大多默认以单次局部闭环为基本尺度，因此难以解释真实复杂任务中反复出现的持续性失稳。本文提出“长程实践”这一结构概念，将其定义为存在不可预计算顺序依赖链、必须跨检查点持续推进的实践类型：长程不是时间更久，而是关键信息必须在推进过程中逐步生成。我们从“关键输入为何当前不可得”的逻辑分叉出发，穷举得到认知缺口、媒介强制、决策分叉与外部漂移四种来源，并据此建立由运载前提、6 条逻辑必要条件与 7 条工程补偿条件构成的三层结构下限体系。由此可见，单次能力与长程能力之间存在三段非蕴含链：单次能力强不推出单检查点可完成，单检查点可完成不推出跨检查点持续推进，跨检查点持续推进也不推出单轮 Benchmark 高分；当前评测框架因而在能力尺度上存在根本错位。

关键词：长程实践；顺序依赖链；结构下限；智能体评测；人机协同

Contents

1	引言	5
2	长程实践的严格定义	6
2.1	实践与检查点	7
2.2	静态可折叠与关键输入	7
2.3	形式化定义	9
3	信息当前不可得的四种来源	10
3.1	穷举推导：先问真值是否已确定	10
3.2	认知缺口：未执行则真值未定	12
3.3	媒介强制：真值已定，但进不了当前工作集	13
3.4	决策分叉：分支未提交，则所需信息身份未定	14
3.5	外部漂移：当前所需真值属于未来状态	14
3.6	状态转移：获取关键信息的代价	15
4	长程实践的三层结构下限	16
4.1	运载前提	17
4.1.1	轨道的拓扑极限	17
4.1.2	引擎的物理极限——LLM 的运载底线	18
4.1.3	驾驶员的认知极限——人类的运载底线	20
4.1.4	运载前提的不可替代性	21
4.2	逻辑必要条件：防止连续性在结构上断裂	22
4.2.1	条件 1：目的闭合 [硬断裂]	22
4.2.2	条件 2：价值闭合 [软断裂]	23
4.2.3	条件 3：检查点分解 [硬断裂]	23
4.2.4	条件 4：顺序约束 [软断裂]	24
4.2.5	条件 5：状态传递 [硬断裂]	24
4.2.6	条件 6：未闭合标记 [软断裂]	25

4.3	工程补偿条件：把骨架放进真实世界	27
4.3.1	补偿 1：恢复与回滚	27
4.3.2	补偿 2：隔离与边界	27
4.3.3	补偿 3：独立评测	27
4.3.4	补偿 4：指令遵循补偿	27
4.3.5	补偿 5：能力缺失升级	28
4.3.6	补偿 6：机向人主动充分解释	28
4.3.7	补偿 7：机向人主动充分请示与对齐	28
4.4	断裂类型与条件体系映射	29
5	跨域一般性：结构而非题材在起作用	30
5.1	结构变量与题材变量	30
5.2	分形同构：构筑与执行共享同一约束方程	31
5.3	一个最短的跨域锚点：建筑方案评审	32
5.4	收束	33
6	当单次闭合遭遇长程结构：三类结构性失效	33
6.1	失效 I：信息尚未生成时，用更多上下文代偿	33
6.2	失效 II：用记忆替代状态，用自述替代验证	34
6.3	失效 III：在需要价值裁决的节点，用局部便利替代全局判断	34
6.4	收束：共享的结构根因	35
7	结论与推论	36
7.1	推论：能力尺度的三段非蕴含	36
7.2	本文回收的贡献	37
7.3	局限与边界	38
7.4	后续问题	39
A	受控解剖：一份长程实践的结构标本	40
A.1	原题陈述	40
A.2	标本选择：为什么这道题适合解剖	42
A.3	检查点重建：认知推进的断面	43
A.4	断裂定位：四种来源在封闭系统中的具体显现	46

A.5 连续性修复：承载结构如何自然长出	47
A.6 为什么裸连模型会稳定失手	49
A.7 标本的边界：覆盖什么，不覆盖什么	50
B 致谢	50

1 引言

在当代大语言模型（LLM）与智能体（Agent）工程中，一个反复出现的现象是：许多系统在单次指令响应、短上下文推理和局部代码修改上表现良好，但一旦进入真实的长程任务，例如跨周期代码库迁移、持续性文献调研或数周级软件维护，便会出现系统性失稳。即便暂时假设执行者拥有近乎无穷的局部推理能力，这种落差也不会自动消失，因为问题首先不在“智力够不够”，而在任务的关键信息不会在起点同时给出，而是必须在跨检查点推进中逐步生成。长程失败首先是结构问题，而不是智力问题。

这一点可以通过两种极端情形看清。翻译一本 1000 页的小说，工作量很大，但通常不是长程实践，因为每一页在信息上大体独立，可以拆成大量并行的单次任务。诊断一台“按下开关后电脑不亮”的机器，工作量未必大，却天然具有长程结构：即便假设维修者拥有无穷智能，在真正刺探环境之前，他也不知道问题究竟是电源线松动、插排未通电、显示器链路异常，还是主板故障；而每一次检查得到的新观察，又会决定下一步该做什么。长程不是工作量大，而是关键信息当前不可得。

这揭示了现有讨论范式的一个盲区。我们大多默认以“单次局部闭环”作为衡量智能体能力的基本尺度，因此当长程任务失败时，常见反应是继续增加上下文、prompt 链、外部检索（RAG）或工具接入。但这些补丁默认了同一个前提：任务所需的关键信息在起始时刻已经存在，剩下的只是检索、组织和执行。定义 2.4 将表明，当任务存在不可预计算的顺序依赖链时，步骤 $i+1$ 的关键输入 K_{i+1} 必须在步骤 i 执行后才会生成或获得；问题因此不是如何在固定信息池里找答案，而是如何在行动与新信息不断相互生成的链条中维持连续性。

本文的主要贡献如下：

1. **定义贡献：**提出“长程实践”这一结构概念，将其定义为存在不可预计算顺序依赖链、必须跨检查点持续推进的实践类型，并给出检查点序列 $\mathcal{C}$ 、当前信息集 $\mathcal{I}_i$ 、关键输入 K_{i+1} 与顺序依赖的最小形式化定义 (§2)。
2. **来源贡献：**从“关键输入为何当前不可得”出发，给出认知缺口、媒介

强制、决策分叉与外部漂移四种互斥且穷尽的来源，并说明它们为何是结构上不可回避的分叉 (§3)。

3. **下限贡献**：提出由运载前提、6 条逻辑必要条件与 7 条工程补偿条件构成的三层结构下限体系，说明长程实践的可维持性依赖哪些不可替代的层级约束 (§4)。
4. **普适贡献**：说明上述定义与下限框架不依赖 coding 场景本身，而适用于一切存在顺序依赖链的连续实践 (§5)。
5. **解释贡献**：以本文的定义透镜对准当前长程系统中反复出现的三类结构性失效——上下文增大无法代偿未生成的真值、记忆替代结构化状态、局部便利替代全局价值裁决——展示它们共享同一结构根因 (§6)。
6. **尺度贡献**：证明单次能力与长程能力之间存在三段非蕴含链，即单次能力强 $\nRightarrow$ 单检查点可完成 $\nRightarrow$ 跨检查点持续推进 $\nRightarrow$ 单轮 Benchmark 高分，并据此指出当前评测框架的能力尺度错位 (§7)。

本文在正文中仅使用最少的具体锚点；附录 A 提供一份完整受控案例作为经典“解剖样本”——以一道输入输出可机械判定、闭合标准严格、探索轨迹完整留痕的组合数学问题为标本，展示长程实践的结构断裂与连续性修复如何在真实任务轨迹上显形。该标本的完整原始研究资料（包括全部检查点执行轨迹、证伪历史、锚点文件等）已开源至 https://github.com/jopsammy/task1_raw_repo。

2 长程实践的严格定义

本文讨论的不是抽象的“任务很长”，而是一类具有明确结构特征的连续实践：执行者围绕某个局部目的推进任务，但下一步所需的关键信息并不总能在当前工作集中一次备齐。为避免把“长程”误解成“步骤更多”或“工作量更大”，本节先给出最小本体前提，再用可判定的符号对象界定何时一个过程真正进入长程实践区间。

2.1 实践与检查点

实践在本文中不泛指任何执行行为，而指一个执行者（人、AI，或人机复合体）围绕局部目的持续经历“感知 → 判断 → 动作 → 保存中间状态”的闭环推进过程。只要这个闭环还在继续，系统就不是一次性吐出答案，而是在与环境的碰撞中不断修正后续步骤。

因此，“long horizon”之后必须接“practice”。前者只描述外显跨度，后者才指出结构本质：后续路径不是在起点就全部给定，而是在推进过程中逐步被确定。如果一个任务虽然长、虽然大，但全部关键前提在起点就已经稳定存在，那么它只是大规模单次闭合；只有当后续步骤的关键输入不能被当前局部闭环一次折叠时，它才进入本文讨论的长程实践。

定义 2.1 (实践实例与检查点). 一个实践实例记为

$$\Pi = (\mathcal{A}, g, \mathcal{C}, \{\mathcal{I}_i\}_{i=0}^n),$$

其中 $\mathcal{A}$ 表示执行者， g 表示当前局部目的， $\mathcal{C} = (c_0, c_1, \dots, c_n)$ 表示有限检查点序列。检查点 c_i 不是单个原子动作，而是一次局部闭环暂时结束、系统可以稳定停靠并继续推进的位置； $\mathcal{I}_i$ 表示在 c_i 时刻可被稳定调用的当前信息集。

2.2 静态可折叠与关键输入

定义检查点的目的，不是给步骤标号，而是为了判定：下一步所需的材料是否已经包含在当前信息集中。下一定义给出这一判据的精确形式。

定义 2.2 (静态可折叠). 对任意检查点 c_i ，若某信息对象 X 无需改变外部世界状态、无需执行新的探测动作、无需承诺某条尚未选定的分支，仅凭 $\mathcal{I}_i$ 中已掌握的材料通过有限内部计算即可获得，则称 X 对 $\mathcal{I}_i$ **静态可折叠**，记为

$$X \in \text{Fold}(\mathcal{I}_i).$$

定义 2.2 刻意排除了三类常被混进“当前已知”的东西：其一，必须先做实验或刺探才能生成的反馈；其二，必须先选定某条分支后才知道要取

哪一类信息；其三，必须等待外部世界进一步演化后才成立的真值。它们都不能被算作当前局部闭环中可直接折叠的输入。

借助上述对象，可以更精确地区分两类过程。若后续步骤所需的关键信息始终都属于当前工作集的静态折叠范围，系统虽然可能工作量巨大，但仍属于单次闭合的扩展执行；若某一步之后的关键输入不能在当前检查点被静态折叠，系统就必须跨越检查点推进。

定义 2.3 (关键输入与顺序依赖). 对从 c_i 到 c_{i+1} 的推进，若存在一个最小信息对象 K_{i+1} ，使得执行或验证 c_{i+1} 的动作必须以它为前提，则称 K_{i+1} 为该转移的**关键输入**。若

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

则称 c_{i+1} 对 c_i 存在**顺序依赖**，记为 $c_i \mapsto c_{i+1}$ 。

定义 2.3 的重点不在于“后一步依赖前一步”这一常识，而在于它把依赖的判据收紧为：后一步所需的最小关键输入，是否能在当前检查点被静态折叠。只有当答案是否定时，前一步才不是简单铺垫，而是一个真正的信息生成或信息定形事件。

下面用两个极端例子划出边界。它们刻意把工作量与结构特征拆开。

例 2.1 (翻译一部极长的小说). 翻译一部极长的小说工作量极大，但它通常不构成长程实践。原因不是任务小，而是每一页翻译所需的原始材料在起点就已给定；第 n 页的关键输入并不依赖第 $n-1$ 页执行后才会生成的新反馈。对页面级子任务而言，其关键输入可在起始时刻落入某个当前信息集的静态折叠范围，因此这是大规模并列执行，而不是不可预计算的顺序依赖链。

例 2.2 (诊断“按下开关后电脑不亮”). 诊断“按下开关后电脑不亮”的故障工作量很小，但它必然构成长程实践。第一步是否是电源问题，必须靠实际检查后产生的反馈决定；第二步去查显示器、主机还是主板，也必须以前一步新生成的状态为依据。即便假设一个无穷智能的执行者存在，他在碰撞环境之前也无法预知这些反馈，因为这些反馈不是藏在某处等待检索的既有答案，而是由实际探测动作生成的结果。

例 2.1 与例 2.2 共同说明：长程实践的核心不在于步数、时长或算力需求，而在于存在不可静态折叠的关键输入。一旦这种输入出现，路径就不再是“起点已知、后续只需执行”，而变成“执行本身会改变下一步所需的信息条件”。

（附录 A 提供了一个更系统的对照标本：在一个纯形式系统中，闭合标准完全可机械判定，但关键输入仍沿检查点逐步生成——长程性在此不来自外部噪声，而来自任务内部的信息结构。）

2.3 形式化定义

据此，可以把长程实践写成一个最小判定句。

定义 2.4 (长程实践区间与长程实践任务). 若对某个检查点转移 $c_i \rightarrow c_{i+1}$ ，其关键输入 K_{i+1} 满足

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

则称该转移进入一个**长程实践区间**。若一个实践实例 Π 的执行路径中至少存在一个长程实践区间，则称该实例为**长程实践**。

定义 2.4 直接给出了单次复杂任务与长程实践的边界。单次复杂任务可以很长、很重、很难，但只要其关键输入始终可被当前信息集静态折叠，它就仍然属于单次闭合的工程放大；长程实践则不同，某一步之后的关键输入不在当前工作集的折叠闭包之内，系统因此必须跨检查点持续推进。这里的约束不是“模型还不够聪明”，而是“当前可得的信息集合对下一步并不封闭”。

需要强调的是，本文并不主张所有任务都属于长程实践。相反，定义 2.4 的价值恰恰在于把边界收紧：只有当不可静态折叠的关键输入真实出现时，后文的结构下限才被触发。若任务始终停留在单次闭合范围内，那么常规的单次执行能力就已足够，无需额外引入长程承载机制。

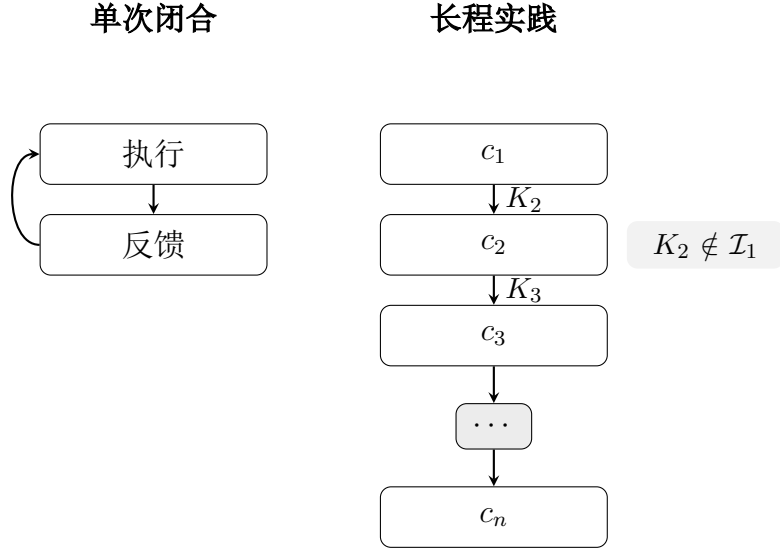


Figure 1: 单次闭合与长程实践的对比。左：所有信息在起始时刻可得，一次闭环完成。右：后续检查点的关键输入 K_{i+1} 在当前信息集 $\mathcal{I}_i$ 中不可得，必须跨检查点推进。

3 信息当前不可得的四种来源

定义 2.4 把长程实践写成一个可判定条件：在某个检查点 c_i ，下一步所需的关键输入 K_{i+1} 不属于 $\text{Fold}(\mathcal{I}_i)$ 。但这一定义本身还没有回答另一个问题：为什么一个关键输入会落在当前折叠闭包之外？本节的核心做法是追问一个更根层的条件：下一步真正需要成立的真值条件，在当前检查点是否已经稳定确定。

3.1 穷举推导：先问真值是否已确定

对从 c_i 到 c_{i+1} 的推进，仅仅说“需要某个关键输入 K_{i+1} ”仍然不够，因为同一个真值条件可以有不同的具体载体。为此，先定义下一步真正需要被固定的对象。

定义 3.1 (下一步真值需求). 对任意检查点转移 $c_i \rightarrow c_{i+1}$ ，设 R_{i+1} 表示执行或验证 c_{i+1} 之前必须被固定的最小真值条件。任何能够稳定见证 R_{i+1} 的具

体数据、记录、反馈或环境读数，都可视为该转移的关键输入 K_{i+1} 的一个实现。

定义 3.1 的作用，是把”要什么信息”与”这个信息由什么载体承载”区分开来。于是， $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的原因只可能来自两个根节点：

1. **真值尚未稳定确定**：在 c_i 时刻， R_{i+1} 本身还没有被固定。
2. **真值已经确定，但当前无法稳定导入**： R_{i+1} 已经固定，但其见证不能在当前媒介条件下稳定进入 $\mathcal{I}_i$ 。

第一类再按”谁决定了 R_{i+1} 的最终形态”继续分叉，就得到三种彼此不同的机制：当前动作反馈决定它，是认知缺口；待选分支决定它，是决策分叉；外部世界的后续演化决定它，是外部漂移。第二类对应媒介强制。由此形成以下命题。

命题 3.1 (信息当前不可得的四种来源)。在采用最小首因归类的前提下，任意一次 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的情形，必可归入下列四类：

1. **认知缺口**： R_{i+1} 由当前步骤执行后产生的反馈决定，未执行则真值未定。
2. **决策分叉**： R_{i+1} 由一个尚未提交的中间分支决定，分支未选定则所需信息身份未定。
3. **外部漂移**： R_{i+1} 取决于外部世界在后续时刻的真实状态，因而在 c_i 时刻尚未稳定坍缩。
4. **媒介强制**： R_{i+1} 已经确定，但其见证在当前媒介、带宽、注意力或窗口预算下无法稳定导入 $\mathcal{I}_i$ 。

命题 3.1 将外部漂移明确放在”真值尚未稳定”一侧，而非”已存在但不可达”。当下一步真正需要的是真实世界在 t_{i+1} 时刻的状态时， t_i 时刻拿在手里的旧快照即便真实、可达、可检索，也仍然不是所需真值本身——它只是一个会过期的历史样本。漂移问题因此首先是真值未定，而不是通路受限。

为避免复合任务中表面上的重叠，本文采用最小首因归类：若多个机制同时存在，则按最早阻止 R_{i+1} 固定或导入的那一层归类。如此处理后，四类在分析口径上互斥，在机制空间上穷尽。

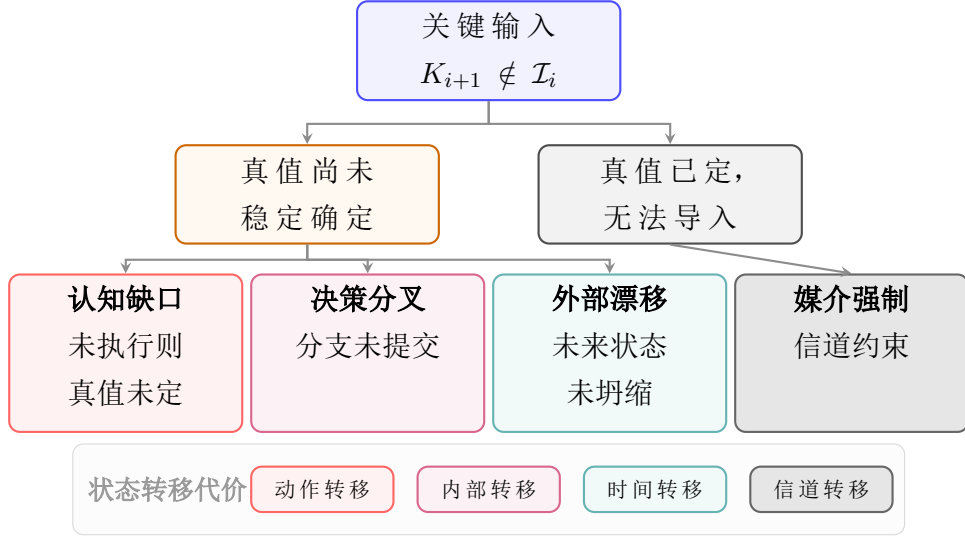


Figure 2: 信息当前不可得的四种穷举来源。根节点第一层分叉：真值是否已经稳定确定。真值尚未稳定确定一侧按“谁决定了 R_{i+1} 的最终形态”继续分叉为认知缺口、决策分叉与外部漂移；真值已定一侧对应媒介强制。

3.2 认知缺口：未执行则真值未定

认知缺口对应的是： R_{i+1} 必须由当前动作与环境碰撞后产生的结果来决定。形式上，

$$R_{i+1} = \phi(Y_i),$$

其中 Y_i 是从 c_i 推进时通过探测、实验、运行或交互才生成的结果；在动作发生之前， Y_i 并不属于当前信息集，因而 R_{i+1} 也无从固定。

定义 2.4 中“诊断电脑不亮”的例子正是如此：是否通电、显示器是否亮起、主机是否自检，都不是在检查之前已经藏在某个工作集里的答案，而是检查动作本身的结果。未执行则不知结果，这个“不知”不是主观无知，而是客观上尚未形成对应真值。

在附录 A 的受控标本中，这一机制反复出现：某条候选路线是否可行，其真值不在起点给定——必须等到穷举验证脚本跑完、对照真实数据之后才能被固定。未执行则真值未定，这正是认知缺口的纯粹形态。

认知缺口的要害在于：不可预知性不来自实践者的智力不足，而来自信息本身的生成时序。只要真值还没被世界产生，再强的执行者也无法把它提前折叠进当前工作集。

3.3 媒介强制：真值已定，但进不了当前工作集

媒介强制描述的是另一种情形： R_{i+1} 已经固定，世界里也确实存在它的见证，但这些见证在当前媒介条件下不能被稳定装载进 $\mathcal{I}_i$ 。可形式化写成：存在某个见证 K_{i+1}^* 使得 $K_{i+1}^* \models R_{i+1}$ ，但在当前传输算子 τ_i 下，

$$\tau_i(K_{i+1}^*) \not\subseteq \mathcal{I}_i.$$

这里的失败可能来自带宽、窗口、噪声、注意力衰减、检索成本或载体切换成本。

媒介强制并非 LLM 独有。人类的工作记忆同样受限于注意力上限和记忆容量：数据库、笔记本、邮件存档都可能保存着正确线索，但”被保存”不等于”已进入当前工作集”。一旦检索、加载和重新理解本身需要额外步骤，媒介强制就已经出现。这一约束的更根层论述来自 Liu 等人^[1]关于智能体级别的分析：现实层面不可能存在欧米伽智能体——一个能够无成本、无延迟、无损耗地访问全部已存在信息的智能体。媒介约束是客观的物理限制，而非特定技术的暂时短板。

LLM 的上下文窗口限制是这类机制最直接的例子。Liu 等人^[2]在多文档问答与键值检索任务上的受控实验已确认了这种“存在但不可得”的系统性：即便对显式长上下文模型，位于上下文中间的关键信息检索准确率也会从首尾的 70% 以上降至不足 50%，且这一 U 型衰减随窗口增长而加剧——窗口大小不等于信息可及性。某条定义明明写在输入最前面，但在长上下文中却不能被稳定调用；信息客观存在，却不能以足够保真的方式进入当前决策位置。

因此，媒介强制不是”信息不存在”，而是”真值虽定，通路受限”。这与认知缺口、决策分叉、外部漂移都不同：后三者的问题发生在真值还没固

定之前，而媒介强制的问题发生在真值已经固定之后。

3.4 决策分叉：分支未提交，则所需信息身份未定

决策分叉的独特之处在于：问题不是拿不到某条已知信息，而是还不知道接下来到底需要哪一类信息。设 B_i 是在 c_i 处尚未提交的中间分支变量，且

$$R_{i+1} = R_{i+1}^{(B_i)}.$$

只要 B_i 还未确定，下一步需要被固定的真值条件本身就没有唯一身份，因此任何具体的 K_{i+1} 都无法在当前检查点被完整指定。

例如，在”重写模块”与”沿旧架构打补丁”之间尚未做出决策时，选择 A 之后要验证的风险、要补的数据、要扫描的依赖与选择 B 之后完全不同。又如，调试时若日志里已有线索则走路径 A，若日志空白则走路径 B；在检查日志之前，执行者甚至不知道应该为哪条路收集后续材料。

决策分叉也不是”不够聪明所以没提前想全”。即便一个系统能够枚举全部候选分支，为每一条分支都预收集并验证其后续信息，所需总成本也会随连续分叉点数快速膨胀—— k 个连续二分决策点产生 2^k 条候选路径，其中绝大部分最终并不会被实际执行。分叉的难点不在枚举本身，而在于未被选中的路径也要求真实的信息获取和验证成本。

附录 A 的记录中，路线 A 至 L 的连续分叉与封禁正是这一机制的直接显现：每条被排除的路线都消耗了真实的探索与验证成本，而排除结论本身也必须被稳定保存为后续搜索空间的永久边界。

3.5 外部漂移：当前所需真值属于未来状态

外部漂移的关键不是”旧信息坏了”，而是下一步真正需要的真值对应的是未来时刻的外部状态。设外部世界状态为 $W(t)$ ，若

$$R_{i+1} = \psi(W(t_{i+1})),$$

且 $W(t)$ 在 $[t_i, t_{i+1}]$ 区间内独立于执行者继续演化，那么即便 $\psi(W(t_i))$ 在当前完全可得，它也不能替代 $\psi(W(t_{i+1}))$ 。此时缺的不是检索动作，而是未来真实状态尚未稳定。

这就是为什么第三方库版本变更、开放网页内容更新、协作文档被他人修改、市场价格波动、文献库新增论文，都会把系统重新拖回长程链条。你在 c_i 时刻拿到的信息可能是对的，但它对 c_{i+1} 未必仍然对，因为下一步真正需要的是到那时仍然成立的真值，而不是过去某刻的正确快照。

因此，外部漂移与媒介强制必须严格区分。媒介强制中，所需真值已经固定，只是载体进不来；外部漂移中，所需真值本身还没有在当前时刻稳定下来。前者是通路问题，后者是时序问题。

3.6 状态转移：获取关键信息的代价

上述四种来源看似分属不同的因果机制，但它们共享一个更深层的结构事实。回顾 定义 2.2 的判据：某信息对象能否仅凭现有材料通过内部计算获得？这个定义本质上在“静态计算”与“状态转移”之间划出了一条界线。线内，信息可以通过前馈计算获得；线外，信息必须以状态转移为代价才能被生成或导入。

四种信息当前不可得的来源，恰好对应了四种必须发生的状态转移：

来源类型	必须付出的状态转移	本质
认知缺口	动作状态转移	执行探测，与环境碰撞产生反馈，真值才被生成
决策分叉	内部状态转移	放弃其他分支、坍缩到唯一路径，所需信息身份才得以确立
外部漂移	时间状态转移	等待世界自身演化到对应时刻，真值才稳定坍缩
媒介强制	物理/信道状态转移	耗费能量或动作将已定真值从冷存储搬运进当前工作集

这四种状态转移的不可替代性来自同一个根层原因：单次闭合范式的底层假设是“信息是给定的，只需要算力去挖掘”。长程实践揭示的相反事

实是——信息不是给定的。它是状态转移的副产物。如果不发生对应类型的状态转移，该信息在当前上下文中在逻辑上就不存在或未定义，而不仅仅是“存在但够不着”。

这就是为什么单次能力强无法弥补长程实践的缺口：前馈计算无法替代状态转移。这不是能力问题，是范畴错误——你试图用一个静态运算器去替代一个必需的因果事件。

这一结论也解释了为什么长程实践总会对承载系统提出额外要求。既然状态转移不可替代，系统就不能只靠一次性推断能力来维持全程推进，而必须在架构上为这四种状态转移提供工程承载：动作状态转移要求持续的探测与验证循环，内部状态转移要求分支的显式提交与边界保存，时间状态转移要求对外部状态的重采样与时效判定，信道状态转移要求跨媒介的信息搬运与保真校验。这些要求不是“最佳实践”的锦上添花，而是任务在信息不封闭的条件下被迫长出的结构骨架——它们构成了下一章将要展开的三层结构下限的逻辑起点。

4 长程实践的三层结构下限

定义 2.4 定义了长程实践的关键特征——检查点之间的关键输入不在起点给定；§3 证明了这种断裂来自认知缺口、媒介强制、决策分叉与外部漂移四类结构性来源，不是偶然。既然断裂带不可消除，问题就变成：一个系统至少要满足什么，才有资格在这些断裂带上持续推进？

本章的答案不是一张平铺的清单，而是严格分层的三层体系。第一层运载前提回答的是准入边界——不满足则不应使用，任何工程补丁无法将它补成可承载系统。第二层六条逻辑必要条件回答的是结构骨架——在准入成立之后，连续性如何不被逻辑性地掐断。第三层七条工程补偿条件回答的是噪声缓冲——在真实人机环境中，如何降低高概率失败。三层不可互相替代：补偿层只能加固骨架，不能替代骨架；逻辑层只能定义连续性，不能绕过准入边界。

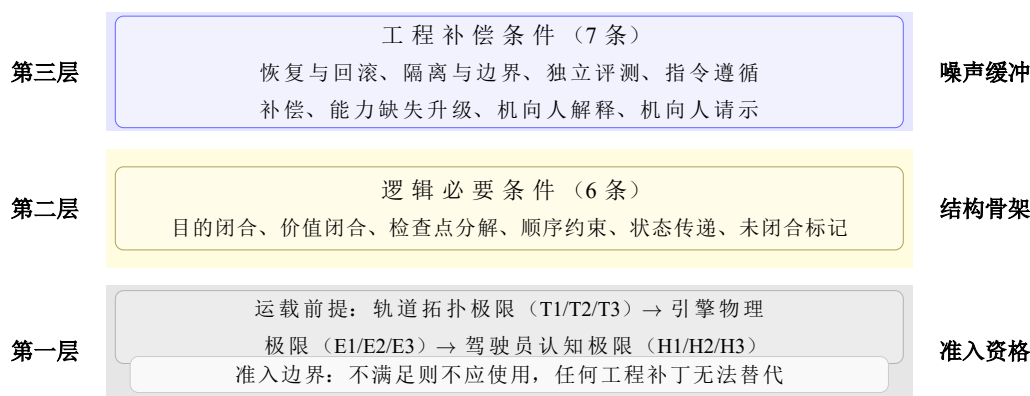


Figure 3: 长程实践的三层结构下限。第一层运载前提定义承载资格（轨道拓扑 → 引擎物理 → 驾驶员认知的三层演绎），第二层逻辑必要条件提供跨检查点的结构骨架，第三层工程补偿条件在现实噪声中降低高概率失败。各层不可互相替代。

4.1 运载前提

运载前提不讨论“做得够不够好”，只讨论“有没有入场资格”。一个系统即便在单次 benchmark 上极强，也仍可能不具备长程资格——benchmark 衡量的是信息完备、路径已知、可单次闭合的任务；长程实践要求的却是在信息尚未齐备、路径必须逐步发现、执行中会反复中断的条件下保持连续性。两者不是同一能力尺度。

承载者不是抽象的“模型”，而是由人、LLM 与工具载体共同组成的联合认知系统。运载前提以三层演绎的方式展开：先定义长程实践轨道的拓扑要求——任何执行者都必须满足；再从轨道要求推导 LLM 必须满足的物理极限；最后从轨道和引擎的约束推导出人必须满足的认知极限。三层逐级推导，缺一不可。

4.1.1 轨道的拓扑极限

轨道拓扑极限回答一个问题：一条不断裂的长程实践链，在结构上必须满足什么？这些要求与谁来执行无关——纯人类团队、纯 LLM、或人机联合都需要满足。

T1：断面可重入性。系统必须能从一次中断后的断面状态——一份被

持久化的上下文快照——重新启动，并正确地继续执行，而不是从零开始或从错误状态开始。长程实践必然经历中断：媒介强制使上下文窗口被清空，人类注意力耗散使人离线。如果每次中断后只能从零重来，长程实践就退化为“必须一口气跑完否则全盘崩溃”的单次任务——这不是长程，这是拉长了时间的单次。缺失它时，实践在物理上无法续接，属于硬断裂。

T2：上下文内自检验性。在任意检查点，系统必须能够在当前可得的上下文中——不引入超出当前信息集的外部信息——自行检验输出是否满足预设的正确性标准。设检查点 c_i 的检验函数为 $V_i : (O_i, I_i) \mapsto \{\text{pass}, \text{fail}\}$ ，T2 要求 V_i 的判定仅依赖 I_i ：任意两个具有相同上下文 I_i 的检验场景， V_i 必须给出相同判定。如果检验 c_i 的正确性需要依赖未来信息（“等后面做完再回来看看对不对”），那么 c_i 的闭合判定在发生时就不可靠——假闭合在依赖链上传播，且无法被当场检测，属于软断裂。

T3：因果链透明性。检查点之间不能形成知识黑洞。后来的执行者必须能够重建“从上一个检查点到这里为什么这样走”的因果链。如果因果链断裂，后续执行者拿到的是一个孤立结果——不知道它基于什么假设、排除了什么替代方案、有什么已知风险。“接续”就变成了盲目的追加而非有判断的推进。缺失它时，后续检查点的判断缺乏信息基础，方向可能漂移，属于软断裂。

这三条轨道要求构成一个执行闭环：T1 保证能接上，T2 保证接上之后能自检，T3 保证接上的人知道前因后果。三者任意缺失，轨道本身就不具备承载长程的物理结构。

4.1.2 引擎的物理极限——LLM 的运载底线

轨道要求不是空谈——它们直接对承担计算的 LLM 提出了物理约束。这里的“物理”指的不是硬件，而是 LLM 作为信息处理节点在结构上不可逾越的能力边界。

E1：窗口性能跨度 $\geq$ 最小可检验步长。

这是运载前提中最核心理论贡献。先引入三个量：

定义 4.1 (检查点分解). 将长程任务分解为检查点序列 $\{c_1, \dots, c_n\}$ ，其中每个 c_i 包含输入上下文 I_{i-1} 、输出产物 O_i 和闭合标准 R_i 。

定义 4.2 (最小可检验步长 $L_{\min}$). $L_{\min}$ 是能够被独立自检验 (T2) 的最小语义单元所对应的最小有效产出量。若 c_i 的产出粒度小于 $L_{\min}$, 则无法为 R_i 写出独立有效的判定准则——“一行代码对不对”没有局部语义, 必须放在更大的语义单元中才有意义。 $L_{\min}$ 不是常数, 由任务域的最小可验证语义粒度决定。

定义 4.3 (LLM 有效执行跨度 S). S 是 LLM 在单次上下文窗口内, 能够稳定地、可被 rubric 验证地产出的最大有效产出量。 S 不是上下文窗口长度, 而是在该窗口内可稳定产出通过 R_i 的产物的能力跨度。

现在来看两者之间的张力。对于每个检查点 c_i , 其产出量 $|O_i|$ 同时受到两个方向的挤压: 向下, $|O_i| \geq L_{\min}$ ——否则 T2 失效, 检查点不可独立检验; 向上, $|O_i| \leq S$ ——否则 LLM 无法在单次窗口内稳定完成该检查点。

定理 4.1 (E1 不可行性定理). 若 $S < L_{\min}$, 则不存在任何检查点序列 $\mathcal{C} = (c_1, \dots, c_n)$ 使得每个检查点的产出量 $|O_i|$ 同时满足:

$$\forall c_i \in \mathcal{C}: \quad L_{\min} \leq |O_i| \leq S.$$

这一定理揭示了一个二择困局: 当 LLM 的单步性能 S 弱到无法覆盖一个可独立检验的最小语义单元 $L_{\min}$ 时, 系统只能在两条绝路上择一——要么把检查点切到足够窄 (满足 $|O_i| \leq S$), 但切碎后的检查点不可独立检验 (T2 失效, 假闭合传播); 要么保持检查点的检验粒度 ($|O_i| \geq L_{\min}$), 但跨度超出 LLM 能力, 检查点内部崩溃。两条路径都通向长程破产。

这正是 benchmark 无法测出的维度。Benchmark 题目在开始前边界已给定、正确性标准已知、所需信息已经齐备——它测的是“在有明确边界的问题上求解”的能力。而 E1 测的是“在信息不完备的任务中, 每一步的跨度是否足够覆盖一个可检验的语义单元”。两者不是同一尺度——一个单轮表现优异的基座可能在持续执行中逐步偏离契约, 而偏离本身不会被单轮 benchmark 捕获。

E2: 无信息丢失。 LLM 在跨越检查点时, 不能因内部知识缺失或上下文截断而造成信息丢失——包括“之前发生了什么”(外部上下文信息)和“这个领域怎么做”(内部知识覆盖)。T3 要求因果链可以被重建, 如果 LLM 在检查点之间丢失了之前的关键上下文, 后续输出就成了无根之水——因

果链断裂为黑洞。T1 要求从断面重启，如果 LLM 的内部知识在重启后不记得某些领域规范或约定，重入后的执行可能偏离原路径。知识缺失无法通过增加检查点或改进契约来解决——这些手段本身依赖 LLM 能正确理解和使用信息。属于硬断裂。

E3：最低指令遵循。 LLM 必须具备足以让契约、评测、检查点复核、状态交接等工程脚手架生效的最低指令遵循能力。T1 要求系统从快照重启——但如果 LLM 不按规范格式输出快照，后续执行者拿到的就是一个不可解析的断面，T1 在工程上无法实现。T2 要求按 rubric 自检——但如果 LLM 不遵循检验指令，自检结果就不可信。关键的是，AC 范式对 LLM 缺陷的补偿手段——更清晰的契约、更严格的评测、更频繁的检查点复核——本身依赖 LLM 遵循这些补偿指令。如果基座在结构上不具备最低限度的指令遵循能力，AC 的补偿层会**递归失效**：补偿指令本身不被遵循。属于硬断裂，换基座是唯一应对方式。

4.1.3 驾驶员的认知极限——人类的运载底线

引擎满足了物理极限后，检查点可以被推动。但推动的方向对不对，取决于人——而人同样有认知极限。这一判断承接了 Simon^[3] 关于有限理性的经典论述：人的认知资源并非无限，在复杂决策中，个体不寻找全局最优解，而是在信息不完备、计算能力有限、时间压力下寻找满意解。下文的三条人类运载底线并非重新发现这一事实，而是将其从一般性认知约束，转化为长程检查点语境下可操作的三约束。

H1：非对称验证带宽。 人在检查点处“读取并理解状态”的认知成本，必须严格小于其当前可用的注意力预算。令 $C_{\text{verify}}(c_i)$ 为人在检查点 c_i 处的验证认知成本， $B(t_i)$ 为到达 c_i 时剩余的注意力预算（ $B(t_i)$ 随累计时间 t_i 单调非增，即 $B(t_{i+1}) \leq B(t_i)$ ），则必须满足：

$$C_{\text{verify}}(c_i) < B(t_i).$$

当这一不等式被违反时，人会产生**橡胶图章效应**——形式上走过批准流程，实质上跳过判断。此时价值闭合成了一场空转：在工程形式上完成了，在认知实质上没有发生。

这个约束与 E1 的约束形成了检查点设计空间的双重边界。E1 定义了

检查点粒度的下限 ($|O_i| \geq L_{\min}$) 和上限 ($|O_i| \leq S$)；H1 追加了一个时间维度上的衰减约束——即使 $S \geq L_{\min}$ 成立，人的注意力预算也会随时间衰减，可能在某个 t_i 处跌破验证成本。这意味着检查点间距不能仅看 LLM 的能力跨度，还必须考虑人的注意力衰减曲线。

H2：外部现实锚定力。人必须具备将系统的内部一致性映射到外部现实价值的能力——在一组逻辑上自洽的输出中，判断它是否仍然符合外部现实的需求和约束。T2 保证了系统能在当前上下文中自检验“按契约做完了吗”，但它检验不了“做完的这件事还有意义吗”。当发生外部漂移或决策分叉时，LLM 作为封闭的信息处理系统是无感的——它会继续完美地执行一个已经过时的目标。人必须具备跨越“系统内部逻辑 $\rightarrow$ 外部现实”这个鸿沟的认知跳跃能力。如果人在该领域也是外行，他只能看到“没报错”（目的闭合），却看不出“方向已经不对了”（价值闭合失效）。缺失时，应对方式不是增加工程补偿——补偿帮不了外行做判断——而是缩小实践范围或更换判断者。

H3：决策制动与重置阈值。在接收到机器的异常信号、偏离警告或价值请示时，人必须具备克服系统推进惯性、按下停止键或重置方向的心理与判断阈值。T1 和补偿 1 从机制上提供了回退的能力，但“是否回滚”是价值决策——只能由人做。机器有推进惯性：当系统发出信号要求人类裁决时，如果人每次都因为疲惫、犹豫或分析瘫痪而选择“默认继续”，那么回退机制在工程上就形同虚设——系统有回滚能力但从未被触发。分析瘫痪尤其致命：如果人在检查点面对的是开放性问题（“你觉得该怎么做？”），决策消耗的認知能量极大，人倾向于搁置或默认通过。这解释了为什么工程协议需要将开放判断转化为选择题——降低决策的認知成本，确保人在阈值内能完成制动。

4.1.4 运载前提的不可替代性

运载前提是准入条件，不是优化项。对于不具备最低承载资格的系统，增加更多工程补丁不会把它补成可承载系统；补丁只能加固一个已经具备资格的骨架，不能替代资格本身。图 4 给出了三层运载前提的演绎结构及任一层的阻断后果。

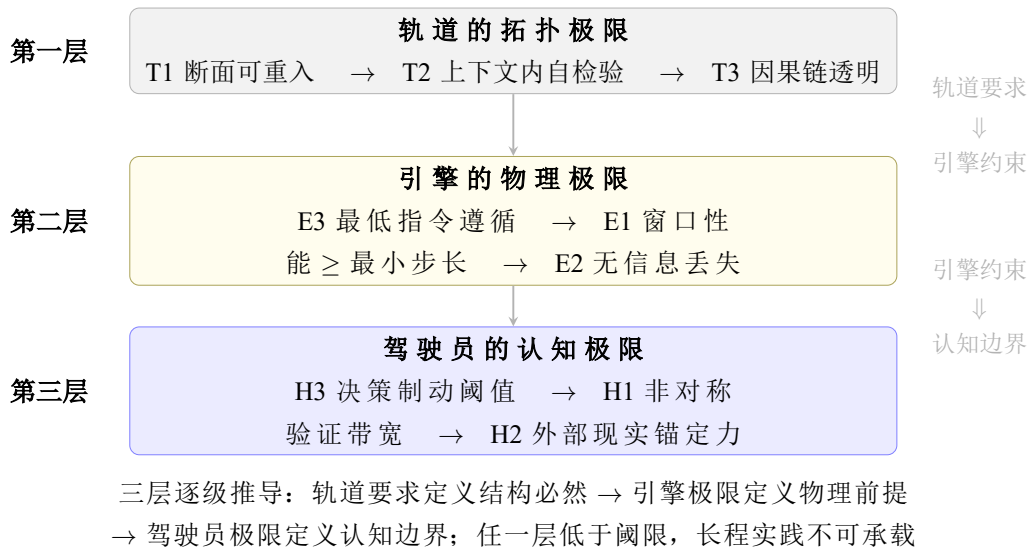


Figure 4: 运载前提的三层演绎结构。第一层轨道拓扑极限定义任何长程实践都必须满足的结构要求（T1/T2/T3）；第二层从轨道要求推导 LLM 作为计算节点的物理极限（E1/E2/E3）；第三层从轨道和引擎约束推导出人作为价值判断者的认知极限（H1/H2/H3）。三层逐级推导，缺一不可。

4.2 逻辑必要条件：防止连续性在结构上断裂

在运载前提成立后——即系统具备进入长程实践的最低资格后——六条逻辑必要条件定义连续性如何在结构上被维持。这里的“必要”不是“通常最好有”，而是缺失后会出现可明确定位的结构断裂。每条条件标注断裂类型：硬断裂表示实践在物理或逻辑上无法继续；软断裂表示推进表面上仍可继续，但结果已不可靠。

4.2.1 条件 1：目的闭合 [硬断裂]

根因：实践是目的驱动下的闭环活动。没有“什么算完成、什么算未完成”的闭合标准，反馈循环无法判断是否完成了当前循环。检查点分解和未闭合标记都以此为前提——“未闭合”需要先有“闭合”才能被判定。通俗来讲，就是每一步产出都要有可验收的检查点。

缺失后果：系统可以不断忙碌，却无法区分自己是在推进还是在漂移。不知道何时算完成，就无法定义检查点。

与上下游的咬合关系：目的闭合是检查点分解和未闭合标记的定义前提——没有闭合标准，“未闭合”不可定义，“检查点”失去锚点。价值闭合以层级管控关系作用其上。

对工程实践的指导：Harness 必须在承接任务前强制建立可操作的闭合标准（契约），否则禁止进入执行。

4.2.2 条件 2：价值闭合 [软断裂]

根因：目的天然具有局部性，而局部目的的正确性不能由目的自身判定。需要更高层的价值判断——在 AC 范式下，这个判断权属于人。价值闭合通常在关键检查点由人类介入确认。

缺失后果：推进本身不被阻断——可以做完，但可能做错方向。系统可能非常高效地走向错误方向。

与上下游的咬合关系：价值闭合以层级管控关系作用于目的闭合之上——“做完了”不等于“做得对”。它是补偿 6 和补偿 7 的存在前提：没有价值闭合，机向人解释和请示就失去了功能锚点。

对工程实践的指导：每个关键检查点必须预设人工介入节点，且该节点的触发条件必须基于明确的工程信号而非人类自觉。

4.2.3 条件 3：检查点分解 [硬断裂]

根因：如果整体目标不可分解，“检查点”这个概念就不存在，长程实践退化为单轮赌博——要么一次做完（不是长程），要么永远无法推进。可分解性不等于“人预先拆好”——分解可以由 agent 执行，但可分解性必须客观存在。

缺失后果：长程实践退化为单轮任务——无法分段推进，无法局部验证，无法中断续接。

与上下游的咬合关系：检查点分解一旦成立，顺序约束和状态传递才有了明确的操作对象。分解粒度必须早于资源耗散使执行能力跌破可执行阈限之前——这一设计约束指向运载前提中 E1 和 H1 的双重边界。

对工程实践的指导：任务承接前必须验证整体目标的可分解性——若不可分解，Harness 应拒绝承接，而非试图以单轮方式硬做。

4.2.4 条件 4：顺序约束 [软断裂]

根因：可分解性回答“能不能拆”，顺序约束回答“拆完后如何组织”。如果只知道任务可以分解但不知道各子任务之间的时序依赖关系，并行分工就是不可靠的——两个子任务可能互相等待或彼此覆盖。

缺失后果：推进可继续，但会不断返工、空转。当蕴含严格偏序（前置产出是后续检查点的逻辑输入）时，软断裂可升级为近似硬断裂的死锁——后续检查点在物理上无法启动。

与上下游的咬合关系：顺序约束以检查点分解为前提——没有检查点，依赖关系就没有操作对象。对并行 Harness 而言，它是串并行策略的存在前提。

对工程实践的指导：Harness 应检测并标注偏序关系，对严格偏序型依赖实施阻断性门禁，非严格依赖则标注风险即可。

4.2.5 条件 5：状态传递 [硬断裂]

根因：检查点 N 的闭合结果必须可被检查点 $N + 1$ 获取。但“状态可获取”（如 Git 记录了文件变更）不等于“状态可接棒”——新执行者理解为什么文件是这个状态、哪里还没做完、下一步从哪开始。因此状态传递天然要求语义标注：做到哪里、为什么做到这里、哪些已闭合、哪些未闭合、下一步从哪里接。

缺失后果：后续检查点的初始条件缺失，实践在物理上无法续接。每次交接都像重新开题，之前的判断依据、约束条件和未决项在断面处蒸发。

与上下游的咬合关系：状态传递以检查点分解为前提，并与未闭合标记深度咬合——传递的状态快照必须包含未闭合信息。补偿 6 是其在人-机方向上的工程补足。

对工程实践的指导：Harness 必须在每个检查点闭合时强制执行状态快照，快照必须包含语义标注而不仅是文件内容。无状态传递则禁止下一步启动。

4.2.6 条件 6：未闭合标记 [软断裂]

根因：系统必须能区分“已闭合”“未闭合”“当前不可判定”三种状态，并禁止将后两者记录为“已闭合”。核心不是要求执行者诚实——那是不可靠的——而是要求系统结构能够独立地产生未闭合信号。测试失败、类型错误、契约校验未通过，都是这类信号；“我觉得应该没问题”则不是。

缺失后果：推进可继续——“看起来都完成了”可以一直走下去——但错误的中间状态被当作已完成前提继续向后传递，整条依赖链建立在假闭合之上。

与上下游的咬合关系：未闭合标记以目的闭合为前提——“未闭合”需要先有“闭合”才能被判定。它是补偿 3（独立评测）的逻辑前提——独立评测是未闭合标记在工程上的强制实现方式。

对工程实践的指导：Harness 必须在每个检查点节点强制输出三值状态标注，并禁止执行者自述替代独立证据。

以上六条条件并非平铺并列。目的闭合为检查点分解和未闭合标记提供定义前提；检查点分解一旦成立，顺序约束和状态传递才有了明确对象；价值闭合作为更高层判断贯穿其上，负责在“做完了”之外继续约束“做得对不对”。

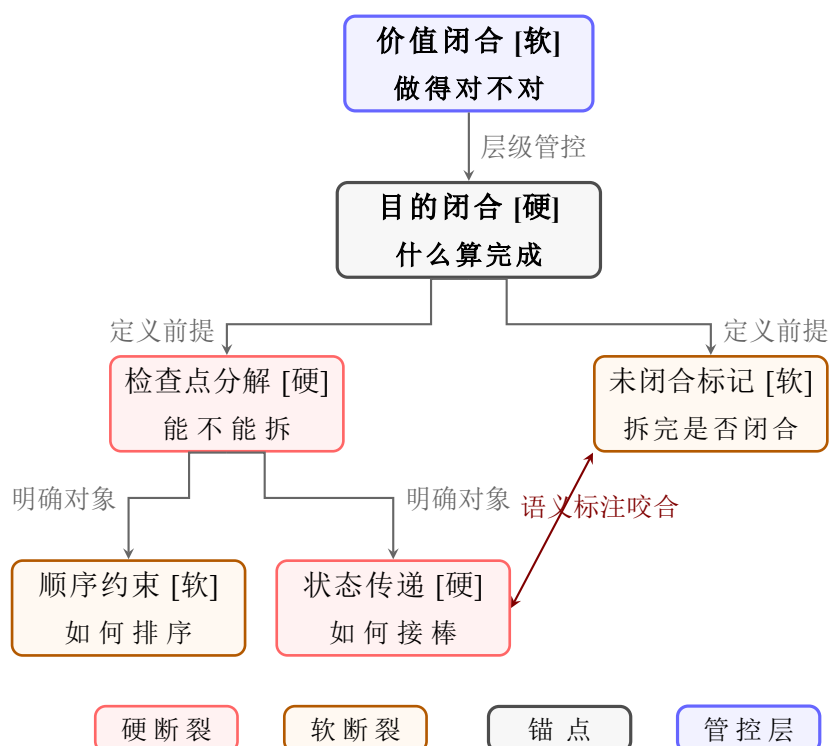


Figure 5: 六条逻辑必要条件的结构化依赖关系。目的闭合为检查点分解和未闭合标记提供定义锚点；检查点分解一旦成立，顺序约束和状态传递才有了明确操作对象；价值闭合以层级管控关系贯穿其上，约束”做得对不对”；状态传递与未闭合标记通过语义标注深度咬合，确保交接处信息不蒸发。硬断裂与软断裂的标注指示缺失后的结构后果。

这六条条件并非抽象分类。附录 A 的受控标本中，它们各自对应真实任务轨迹上的具体对象：目的闭合对应该题不可降级的 GATE 表，检查点分解对应从组合分解到代数降阶的多阶段推进，状态传递对应 spec 与 current-note 之间的双向同步与作废索引维护，未闭合标记对应“17/21 已通过、4 例待攻克”的三值状态标注，价值闭合对应“禁止重试路线”清单作为人类裁决的外置边界。这些对象不是事后整理出的最佳实践，而是任务在断裂带上存活时被迫长出的结构。

4.3 工程补偿条件：把骨架放进真实世界

即便运载前提和逻辑层都成立，真实实践仍会被各种高概率噪声破坏：执行会偏航，错误会外溢，评测会被执行者自述替代，人也会在关键节点失去知情或错过介入时机。工程补偿条件要处理的正是这些“逻辑上可做、工程上易坏”的部分。它们不是运载前提的替身，也不是逻辑必要条件的替身，而是让下限体系在真实环境里真正可运行的装甲层。

4.3.1 补偿 1：恢复与回滚

当闭合判断出错、执行偏航或环境变化时，系统必须能回到已知有效状态。一次闭合判定错误不应永久污染整条实践链。但回滚的决策权必须属于人——回滚本身是一步不可逆的决策，这个决策的合理性只能由价值闭合层判断。它主要加固状态传递与未闭合标记：当错误被发现后，系统需要有把错误隔离在可恢复的边界内，而非只能硬着头皮继续。

4.3.2 补偿 2：隔离与边界

系统必须限制错误传播半径，避免一次局部失误污染整个实践单元。检查点即便被分解出来，如果没有边界约束，一个错误修改就可能把多个后续检查点一并污染。隔离不是替代检查点分解，而是限制一次失误的传播半径，让错误停留在局部、可诊断、可回滚的范围内。

4.3.3 补偿 3：独立评测

验收不能依赖执行者自述。必须由测试、检查器、类型系统、人工验收或其他独立证据链承担。它是未闭合标记在工程上的强制实现方式——保证未闭合信号不依赖于执行者的自我报告，而是来自一个可以独立否证的机制。

4.3.4 补偿 4：指令遵循补偿

“信息到达”不等于“信息被按原意执行”。当执行者（尤其是 LLM）天然倾向于用自身偏好覆盖外部意图时，仅靠写下规则和契约是不够的——需要通过契约显式化、检查点复核和偏离检测等机制，持续校正“收到指

令”和“按指令执行”之间的落差。它与运载前提 E3 形成对称：E3 解决的是“基座是否有能力遵循”，补偿 4 解决的是“有能力的基座在执行中是否会偏离”。

4.3.5 补偿 5：能力缺失升级

当执行者本身不具备完成某一步所需能力时——如 LLM 对某个语言、框架或范式的知识完全空白——系统必须允许显式上交、换手、重新分配或引入外部能力，而非鼓励其用幻觉把缺口填平。与未闭合标记的区分：未闭合标记处理的是“知道某事未完成”（已知的未知），能力缺失处理的是“不知道某事需要知道”——执行者甚至不理解任务意味什么。升级机制的有效性依赖于一个前提：系统内存在能够接手的能力上位方。如果知识缺失是全局性的（LLM 不知道、人也不懂、无外部工具可用），则任务超出当前系统的运载上限——AC 可以暴露缺口并阻止假装推进，但不能凭空补全知识。

4.3.6 补偿 6：机向人主动充分解释

人在检查点做出价值判断的前提是理解当前状态。但 LLM 天然倾向于省略决策理由、跳步或使用人难以追踪的推理路径——这造成了一种结构性信息不对称：人在检查点面对的是一个“已经变成这样的状态”却不知道“为什么变成这样”。系统必须要求机器在需要人类介入的关键节点，主动提供最小解释包：当前检查点的完成状态、关键决策及其理由、未闭合项、待判断问题、风险提示。它是状态传递在人-机方向上的工程补足：条件 5 保证了 LLM → LLM 的状态传递，补偿 6 保证 LLM → 人的状态传递。两者缺一不可。

4.3.7 补偿 7：机向人主动充分请示与对齐

LLM 执行者具有一种系统性倾向：在遇到价值模糊、边界条件或自身判断力不足时，倾向于“用自己的偏好填补空白”而非“暴露空白请人决策”。这导致价值偏移可以在多个检查点之间累积而未被察觉。系统必须要求机器在以下情况下主动暂停并请示人类：遇到非技术性的价值取舍、自

身判断力触及边界、发现执行路径与契约发生静默漂移、当前决策可能对后续检查点产生不可逆的路径锁定效应。补偿 7 与补偿 6 共同构成了人-机交互的工程协议层——补偿 6 解决“人到场时是否知情”，补偿 7 解决“是否需要人到场”。两者确保价值闭合虽然在定义上由人负责，但在工程上也有机制确保人真的能履行这一责任。

4.4 断裂类型与条件体系映射

将 §3 的四种信息截断来源与本章条件体系放在一起看，可以得到一个更清晰的结论：不同断裂类型压迫的并不是同一组条件，它们各自沿着不同路径把系统推向失败。图 6 给出了四条主要断裂路径。

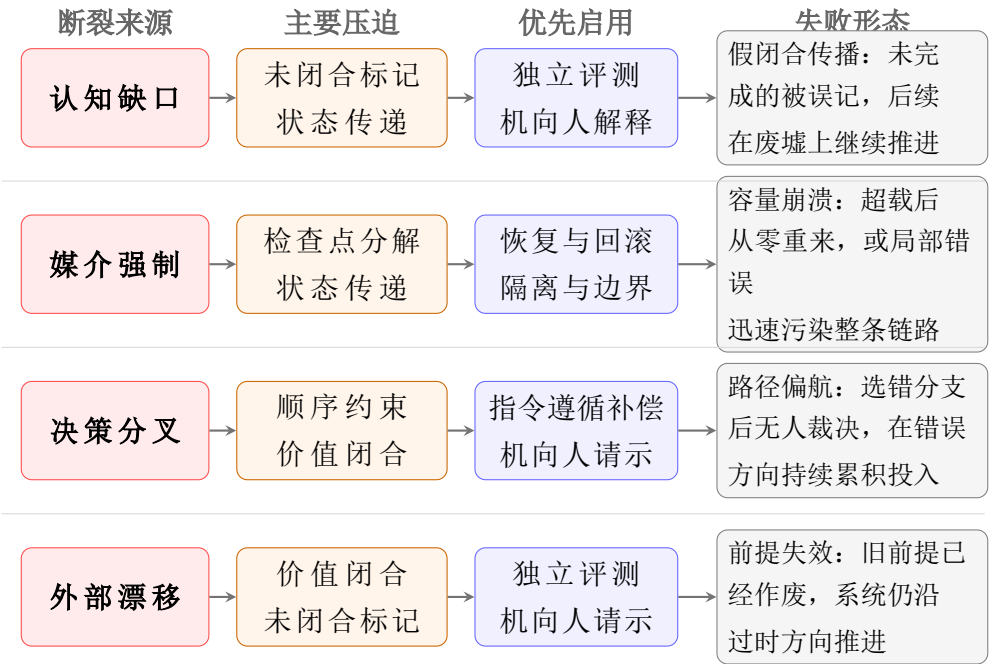


Figure 6: 四种断裂类型各自沿不同路径穿透条件体系。认知缺口和外部漂移共用未闭合标记但分别从不同方向侵入——前者在真值未生成时抢先闭合，后者在真值已失效后拒绝重新打开。媒介强制和决策分叉则分别攻击检查点的粒度约束和顺序约束。四种路径互不重叠，不存在可以统一处理所有断裂的万能补丁。

这个映射有两层含义。第一，不同断裂来源对应的是不同的薄弱面，因

此“给系统多加一点补丁”并不能统一解决问题。第二，工程补偿条件只能加固它所对应的逻辑层，却不能越级替代它。独立评测可以帮助暴露假闭合，但不能替代未闭合标记这条逻辑要求；机向人请示可以确保价值节点不被静默绕过，但不能替代价值判断本身；恢复与回滚可以把偏航控制在已知边界内，但不能把不具备运载资格的系统补成有资格的系统。

5 跨域一般性：结构而非题材在起作用

如果接受 §2 的前提——智能体是感知-行动闭环本身，实践与智能是同一闭环的两个不可分割的相位——那么长程实践的结构要求就不是绑定在 coding 工具或 LLM 架构上的偶然约束，而是绑定在闭环结构本身上的必然约束。本节任务是证明这一点：先区分结构变量与题材变量，再用分形同构展示同一约束方程在构筑与执行两个维度上的实例化，最后以一个非 coding 案例展示分析框架的直接适用。

5.1 结构变量与题材变量

定义 2.4 的判定使用了检查点序列 $C = (c_1, c_2, \dots, c_n)$ 、当前信息集 I_i 、关键输入 K_{i+1} 与顺序依赖关系——这些是**结构变量**：它们描述的是信息如何沿时间轴展开，与任务内容的具体学科无关。相反，“写代码”“做数学”“设计建筑”是**题材变量**：它们描述的是信息的内容域。

长程实践的判定只依赖结构变量：只要存在某个 i 使得 $K_{i+1} \notin I_i$ ，且该依赖不能被预计算折叠，系统就进入了长程实践区间。这个判定不关心 K_{i+1} 是“下一个函数的签名”还是“甲方对方案的真实评价”。同理，§3 的四类来源——认知缺口、媒介强制、决策分叉、外部漂移——描述的是 $K_{i+1} \notin I_i$ 的四种结构性成因，而非四种行业现象。§4 的三层下限——运载前提、逻辑必要条件、工程补偿条件——描述的是跨过这些断裂带所需的最小结构资格，而非某类工程师的最低素养。

因此，声称这套框架具有跨域一般性，是一个可检验的结构命题：**任何存在不可预计算顺序依赖链的领域，其信息结构都会在同样的位置撞上同样的结构问题。**反之，一个任务即便规模巨大、耗时漫长，如果所需信息在

起点已经基本齐备且各部分可并行拆解，它就不是长程实践——信息结构不同，结构问题就不同。

5.2 分形同构：构筑与执行共享同一约束方程

结构一般性最容易被误解的地方在于功能定位。有人接受”设计架构、规划系统”是长程实践，却认为”调试算法、定位报错”只是技术执行，与结构问题无关；也有人反过来承认复杂算法的逐步优化需要跨检查点推进，却把系统架构设计看成一次性智力判断，似乎”做事”才有顺序依赖、”想事”可以在头脑中一步到位。两者都忽略了一点：在长程实践中，构筑与执行不是两类不同的活动，而是在不同功能定位上反复出现的同一信息结构。

这一判断可以直接代入形式变量来检验。取同一个软件项目的构筑侧和执行侧各一条典型轨迹：

- **构筑：**检查点序列 $C^{(\text{构})} = (c_1, c_2, c_3) = (\text{需求分析}, \text{架构设计}, \text{迭代交付})$ 。在 c_1 完成前， c_2 所需的关键输入 $K_2^{(\text{构})}$ （模块边界、接口约束、技术选型前提）尚未进入 $I_1^{(\text{构})}$ ——它们依赖需求分析的产物才能确定。
- **执行：**检查点序列 $C^{(\text{执})} = (c_1, c_2, c_3) = (\text{定位报错}, \text{修改代码}, \text{验证修复})$ 。在 c_1 完成前， c_2 所需的关键输入 $K_2^{(\text{执})}$ （根因而非现象、修改的具体范围）尚未进入 $I_1^{(\text{执})}$ ——它们依赖调试的实际输出和 `stack trace` 的分析结果。

将两条轨迹并置，结构关系的重合立刻显现：

$$\text{构筑： } K_2^{(\text{构})} \notin \text{Fold}(I_1^{(\text{构})}), \quad K_3^{(\text{构})} \notin \text{Fold}(I_2^{(\text{构})})$$

$$\text{执行： } K_2^{(\text{执})} \notin \text{Fold}(I_1^{(\text{执})}), \quad K_3^{(\text{执})} \notin \text{Fold}(I_2^{(\text{执})})$$

两边的 I_i 和 K_{i+1} 承载的**内容**完全不同——一个是需求文档和架构图，另一个是报错信息和代码行号。但 $K_{i+1} \notin \text{Fold}(I_i)$ 这个**约束关系**在两个维度上逐对重合——如同两个系数完全相同、变量名不同的方程组，在代数上是同一个对象。这就是分形同构的精确含义：内容相异，约束方程同构。

这个结论不是类比，而是同一个信息拓扑在构筑与执行两个维度上被实例化了两次。它有两个直接后果。第一，§4 的三层下限不以功能定位为前提：检查点分解、状态传递、未闭合标记、价值闭合这些要求，会在构筑

和执行两个层面分别提出，只是实现形式不同——构筑上可能是里程碑评审和版本基线，执行上可能是测试用例和 diff 审查。第二，长程系统不是“顶层有规划，底层靠临场发挥”；执行层若完全不具备同类结构，构筑层的连续性迟早会被局部断裂反噬—— $K_{i+1}^{(执)} \notin \text{Fold}(I_i^{(执)})$ 如果被忽略，产生的错误就会沿着依赖链向上传导，最终污染 $K_j^{(构)}$ 。

5.3 一个最短的跨域锚点：建筑方案评审

建筑方案评审是一个足够短、足够远离 coding 的例子，可以用来直接展示分析框架的适用性。

设计团队提交方案 A 之后，甲方才说“会议室不够”；团队按反馈改成方案 B，甲方又说“会议室太多，办公面积不够”；方案 C 再次触发“预算超了”的新反馈。这个过程的每一步都遵循同一类信息结构：

- **认知缺口：**甲方在看到方案 A 之前，并不真正拥有“会议室数量是否合理”的判断。 K_{i+1} （甲方对空间分配的真实需求）在方案提交前尚未生成——这是 尚未生成 的典型案例，且与建筑专业知识无关。
- **决策分叉：**团队收到反馈后选择“减少会议室、增加办公面积”的修改方向——但这一决策本身会影响后续甲方产生什么新反馈。 K_{i+2} 的真实值依赖于团队在 c_i 做出的中间决策，而非在起点就已固定。
- **外部漂移：**“预算超了”这条约束在方案 C 提交后才被显式化——不是甲方刻意隐瞒，而是在前两轮中，甲方的注意力被会议室问题占据，预算约束尚未进入有效讨论空间。任务边界在交互过程中持续被改写。

三个断裂类型在同一个非 coding 案例中自然出现，且 §3 的四种来源分类不需要任何跨域适配就直接覆盖了它们。这个案例中没有出现的是媒介强制——但如果评审发生在跨国团队之间，时差、语言转译和异步沟通就恰好会引入媒介强制。换言之，四种来源在非 coding 领域不是全部同时到场，但它们出现的条件与 coding 领域完全相同：只要 $K_{i+1} \notin I_i$ ，就会在四种结构性成因中找到至少一种解释。

5.4 收束

长程实践的结构下限描述的是一种信息结构，而不是某个行业经验。它以结构变量为对象，以 $K_{i+1} \notin I_i$ 为判据，因此任何出现同一信息结构的领域，断裂来源与承载要求都会以分形同构的方式重新出现。表面内容会变：代码、图纸、诊断报告各不相同；约束方程不会变：检查点之间的信息截断及其沿时间轴的传导始终遵循同一结构关系。

正因闭环结构不随智能体的“材质”（人类、LLM 或人机复合体）而改变，§4 建立的条件体系才不以具体题材或具体智能体类型为前提。这是一个结构推导：只要存在 C 、 I_i 、 K_{i+1} 与 $K_{i+1} \notin I_i$ 的关系，断裂来源的四类成因和承载连续性的六条逻辑必要条件就会随之生效，与 C 、 I_i 、 K_{i+1} 的具体内容无关。下一章将展示，当这些条件缺失时，真实系统会以何种可预测的方式失败。

6 当单次闭合遭遇长程结构：三类结构性失效

§2 定义了长程实践的信息结构，§3 解释了四种断裂来源，§4 给出了跨断裂所需的三层结构下限，§5 证明了这一框架不依赖特定题材。一个理论框架在逻辑上自洽之后，还需要展示另一件事：它能如何重新解释读者已经熟悉的经验现象。否则，读者可以承认它逻辑自洽，但不会觉得它有用。

前文的推导链（§2 到 §5）已经完成了证明。本章的任务是用 §2 和 §3 的定义透镜，对准当前长程 Agent 工程中反复出现的三类典型失败，让读者看清：那些看似彼此无关、常被归因于“模型不够强”或“工程不够精细”的问题，在结构上其实是同一件事——单次闭合系统试图在信息尚未封闭的检查点上，用已有材料强行闭合下一步。

6.1 失效 I：信息尚未生成时，用更多上下文代偿

当长程任务失败，最常见的工程回应是增大上下文窗口——把更多历史对话、更多代码文件、更多检索结果塞进当前推理。背后的假设是：失败源于“相关信息没有被模型看到”，因此只要窗口够大，问题就能解决。

§3 区分了四种信息当前不可得的来源。增大上下文窗口，本质上只处

理了其中一种：媒介强制（信息已存在，但载体受限）。但对于认知缺口（信息尚未生成）和外部漂移（信息属于未来状态），再大的窗口也无法把尚未坍塌的真值折叠进来。一段代码运行后的报错信息、甲方看到方案后的真实反馈、下一个实验的测量结果——这些真值在当前检查点尚不存在。把 100 万 token 的历史对话塞进窗口，只是把更多“过去”搬到了眼前，并没有把“未来”提前生成出来。

上下文窗口代偿的是检索广度，而不是时序生成。当真值尚未在世界上出现时，窗口大小与正确性无关。

6.2 失效 II：用记忆替代状态，用自述替代验证

多数 Agent 系统把多轮对话历史当作任务状态的唯一载体：中断后恢复时，把之前的对话记录重新喂给模型，期望它“接上”。与此同时，Agent 输出一句“已完成”或“I have fixed the issue”，系统就默认该检查点闭合，很少引入独立的外部验证。

§2 定义了检查点与当前信息集：检查点闭合需要的是“什么已闭合、什么未闭合、下一步从哪里接”的结构化断面状态，而不是“之前发生了什么”的时序记忆记录。对话历史属于后者——它记录了事件序列，但没有显式标注哪些前提已被验证、哪些仍待确认、当前断面的依赖链状态。对话历史是记忆，不是状态。用记忆代替状态，就像要求下一位接班的工人去读前任的所有日记，而不是交给他一份工程图纸。

§4 的未闭合标记（条件 6）要求系统能独立产生闭合/未闭合信号，而不是依赖执行者的自述。当系统用自述代替验证时，它在做的和失效 I 是同一类操作：把尚未被独立确认的信息身份，当作已闭合的前提向后传递。在长程依赖链中，没有独立验证的自述闭合，本质上是将执行者的局部判断向全局强制注入。

6.3 失效 III：在需要价值裁决的节点，用局部便利替代全局判断

Agent 在遇到阻碍时——编译报错、测试失败、接口冲突——最常见的策略是在局部范围内寻找最快通过当前障碍的方案。删掉一段校验逻辑、绕

过一个类型约束、用一个近似值替代精确计算。每一步在局部看都“合理”，但累积数步之后，系统已经偏离原始目标。

§3 的决策分叉指出：当下一步需要的信息身份取决于尚未做出的分支选择时，系统在结构上无法提前确定该收集什么、该优化什么。当执行者在这种分叉点不等待裁决、而是自行选择局部代价最小的路径时，它实际上是把一个需要外部价值判断才能闭合的分叉，当作一个可以在执行层内部通过技术指标闭合的优化问题。§4 的价值闭合（条件 2）和机向人请示（补偿 7）正是为这类节点设计的——它们要求系统在“做得快”和“做得对”之间不能由执行层自己决定。

每一步的局部合理性，累积数步之后可能等价于全局方向的系统性偏离——而系统在每一步都觉得自己做对了。局部便利可以快速通过当前障碍，但不能替代方向判断。

6.4 收束：共享的结构根因

三类失效在表面上覆盖了不同的问题域——上下文管理、状态交接、路径选择——但在 §2 和 §3 的框架下，它们共享同一个结构根因：单次闭合系统试图在信息尚未封闭的检查点上，用自己已有的材料强行闭合下一步。无论是用更多上下文代偿未生成的真值、用对话记忆代偿结构化状态、还是用局部便利代偿价值裁决，本质上都是在做同一件事——把本属于 c_{i+1} 才应被确定的信息身份，提前在 c_i 处“预闭合”。

这解释了为什么增大窗口、增强检索、优化 prompt 这些局部改进无法根除长程失败：它们都在同一个封闭信息集内部做优化，而没有改变“当前信息集对下一步并不封闭”这一结构事实。

在这个意义上，Claude Code 一类系统在工程迭代中长出的回退与隔离、结构化 handoff、独立验证与主动请示等机制^[4]，不是某家公司的设计偏好，而是独立工程团队在真实约束下被结构压力反复逼向同一组承载设计的自然证据。SWE-bench 从单轮正确率走向真实仓库、真实测试和真实中间反馈的评测演化^[5-6]，以及人机协同研究对混合主动权设计的持续强调^[7-8]，各自从不同侧面收敛于同一个判断：长程能力不是单次能力的加总，而是另一类结构资格。

这不是外部背书，而是旁证——它表明 §4 中的三层下限不是事后整理

出的审美偏好，而是独立系统在长期存活时反复撞上的共同门槛。下一章将从这些结构洞察中收束出关于能力尺度的推论。

7 结论与推论

§2 到 §6 建立了单一论证链：长程实践的形式定义 $\rightarrow$ 关键信息不可得的四种来源 $\rightarrow$ 跨断裂所需的三层结构下限 $\rightarrow$ 跨域一般性的结构证明 $\rightarrow$ 三类结构性失效的解释力。本章不再重复推导，而是从这条论证链中收束出关于能力尺度的推论，回收全文贡献，并明确当前框架的边界。

7.1 推论：能力尺度的三段非蕴含

长程实践不可约化为单次闭合的叠加——§2 从结构上建立了这一点，§6 通过三类失效给出了旁证。由此引出一个关于评测尺度的推论：单次能力与长程能力之间不存在直接的蕴含关系。这里说的不是程度差异，而是尺度差异——它们测量的是不同维度上的结构资格。

命题 7.1 (单次能力强 $\nRightarrow$ 单检查点可完成). 一个模型在单轮或短上下文中表现优异，并不意味着它能稳定闭合一个真实工程检查点。

结构理由：§4 已说明，真实检查点不是把题干换长一点的单轮问题——它要求系统探测自身已知与未知的边界（条件 5），区分哪些约束不能在执行层自行跨越。局部求解正确不等于该检查点在全局约束下可以闭合。

命题 7.2 (单检查点可完成 $\nRightarrow$ 跨检查点持续推进). 即使系统能在局部闭合并交付单个检查点，也不意味着它能在包含多个检查点的任务链中持续推进。

结构理由：§2 定义的顺序依赖决定了：跨检查点后，错误通过交接边界进入后续步骤，不再是一道题答错的问题。状态传递（条件 1）、可逆回退（条件 3）和闭环验证（条件 6）是将局部正确累积为全局正确的结构前提，而它们不能由局部闭合自动满足——§6 的失效 II 已展示了这一点。

命题 7.3 (跨检查点持续推进 $\nRightarrow$ 单轮 Benchmark 高分). 一个具备长程承载结构的系统，在强调瞬时命中率和单轮输出质量的评测中，未必取得与其生存能力匹配的高分。

结构理由：§4 的补偿 7（机向人请示）和 §6 的失效 III 已共同说明：长程系统会主动引入验证、回退与请示机制。这些行为在单轮评测尺度上被计入成本，但它恰恰是系统在信息不封闭条件下维持连续性的结构代价——它们是维持长程连续性的准入条件，而非可选的冗余。

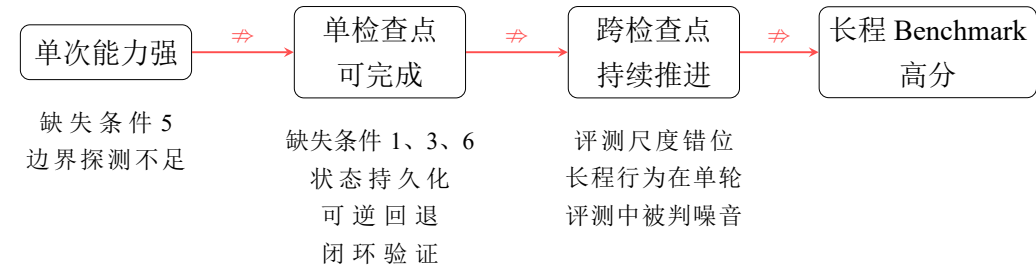


Figure 7: 三段非蕴含链。单次能力强不推出单检查点可完成（缺失边界探测），单检查点可完成不推出跨检查点持续推进（缺失状态持久化、可逆回退和闭环验证），跨检查点持续推进不推出长程 **Benchmark** 高分（评测尺度错位）。

三段非蕴含指向同一个判断：评测长程能力需要长程尺子。当前以单轮命中率和短上下文输出质量为核心的评测体系，测量的是局部闭环中的瞬时表现；而长程实践的核心困难——信息在时间轴上的逐步生成与跨检查点连续性的维持——不在这些尺子的测量范围内。这不是评测做得不够好，而是它测量了不同维度。

7.2 本文回收的贡献

第一，长程实践的最小判定骨架。通过检查点、当前信息集、关键输入与顺序依赖的形式定义，本文把“长程”从工作量描述收紧为一个结构判据：只要下一步所需真值不能在当前信息集中被预闭合，该任务就已经进入长程区间。这一判据的工程意义在于：它把“这个任务到底需不需要承载结构”从一个事后经验判断，变成了一个在 c_i 处就可以执行的形式检查。

第二，关键输入不可得来源的穷举分类。认知缺口、媒介强制、决策分叉与外部漂移是对“为什么 $K_{i+1} \notin I_i$ ”这一逻辑空间的完备划分。§3 的收束段将四种来源统一为同一个原理：前馈计算无法替代状态转移。这个分类的作用是为后续的条件体系提供一个穷举的断裂诊断矩阵。

第三，长程实践的三层结构下限。运载前提（轨道拓扑 → 引擎物理 → 驾驶员认知）回答“有没有承载资格”；6 条逻辑必要条件回答“连续性在结构上如何保证不断裂”；7 条工程补偿条件回答“这套骨架如何在现实噪声中存活”。需要强调：工程补偿层的开放性不是理论缺陷——开放的是补偿实现的工程形式，不是运载资格与逻辑骨架本身的结构判据。

第四，从上述论证链中自然收束出的能力尺度推论。§5 证明了这个框架不依赖特定题材；§6 证明了三类工程失效共享同一个结构根因；本章的推论则收束为三个可引用的命题：单次能力强 $\nRightarrow$ 单检查点可完成 $\nRightarrow$ 跨检查点持续推进 $\nRightarrow$ 单轮高分。三段非蕴含不是对现有评测的否定，而是对当前评测体系测量范围的边界划定。

7.3 局限与边界

第一重局限是逻辑地位。本文给出的是结构下限——一组必要条件。满足这些条件，系统具备了承载资格；不满足，则长程实践会在某个断面结构性断裂。至于满足下限后如何达到更优表现，属于能力上界与策略优化的问题，不在本文的主论证范围内。需要进一步说明的是，本文引入的形式化符号—— C 、 I_i 、 K_{i+1} 、 S 、 $L_{\min}$ 等——目前处于拓扑结构描述层级，尚未给出可直接代入工程测量的物理量化定义（例如 $L_{\min}$ 在一个具体任务中的数值判定方法）。这不是理论的缺陷，而是刻意留出的接口：§2 到 §4 的整套推导表明，这些变量恰恰构成了评测长程能力的必选测量轴——下一代长程 Benchmark 的任务设计、检查点粒度判定与闭合标准，本质上都需要围绕它们来构建。7 条工程补偿条件在这一框架中是“被工程实践逼出的结构近似”，而非已被证明的充分条件——它们的开放性正是这一局限的诚实表达。

第二重局限是验证边界。本文的实证支撑主要来自事后解剖（如附录 A 的受控标本）与独立工程实践的自然收敛（如 §6 的旁证），属于定性的结构病理学诊断。需要说明的是，AC 范式面向 Trae 环境的 v6 工程实现已经完成并在真实长程任务中通过了压测——这构成了前瞻性的工程验证，但该部分不属本文理论论证范围，将以独立技术报告另行呈现。大样本跨任务族的量化因果分析仍有待后续工作，本框架目前提供的是长程实践的结构底座，尚未延伸为可计算断裂概率的动力学模型。

第三重局限是工程摩擦力。本文提出的 7 条工程补偿条件——独立评测、机向人主动解释与请示、状态快照与语义标注、恢复与回滚等——为长程连续性提供了必要的噪声缓冲，但同时引入了显著的执行摩擦力：每个检查点都要求额外的验证步骤、交接记录与审核节点。在短任务或信息已基本闭合的场景中，这种“步步为营”的结构会带来不必要的开销，甚至可能压制大模型基于模糊直觉快速产出结果的优势——它用瞬时执行效率交换了长程稳定性。因此，本框架不应被泛化为所有 LLM 交互的默认标准；其价值在且仅在 $K_{i+1} \notin I_i$ 条件成立时才被激活，在不需要长程承载的场景中强加这套机制是负收益。

第四重局限是适用范围。本文讨论的是存在不可预计算顺序依赖链的任务。当所需信息在起点已经齐备、子问题边界可一次性给定、整条路径可预先折叠时，单次闭环工具已经足够。此时本文框架的作用是“不要误判为长程”的边界澄清——它提供的是一个区分判据，而非在不需要长程承载的场景中强加额外负担。

7.4 后续问题

第一，如何把下限条件落成运行时基座。真正的困难在于把运载前提检测、状态传递、回退、独立验证与请示边界变成联合系统的运行时原语——使它们不再依赖执行者的临场自觉。§6 中旁证的独立工程实践表明这并非空想，但对不同架构（单 Agent、多 Agent、人机复合）的落地方案仍需分别设计。

第二，如何建立面向长程能力的评测框架。本文的论证链已指明方向：若单轮命中率和瞬时输出质量不足以代表长程资格，评测任务就必须显式包含顺序依赖链、未闭合状态追踪、中断恢复与边界探测。更根本地，本文建立的形式变量—— C （检查点序列）、 I_i （当前信息集）、 K_{i+1} （关键输入）、 S （有效执行跨度）、 $L_{\min}$ （最小可检验步长）——直接为这类评测提供了结构蓝图：一个面向长程的 **Benchmark**，其骨架应当围绕“检查点间的信息截断是否被显式建模、关键输入何时被生成或被阻隔、状态传递是否被度量、未闭合标记是否被独立验证”来设计，而非继续在单轮命中率上做精度竞赛。

第三，不同 Agent 架构与大模型基座的组合几乎都需要独立测试与设

计。结构判据是跨域跨模型的——但工程落地不是。不同基座在检查点闭合行为、信息流失模式和指令衰减速率上差异显著，同一套补偿条件在不同组合下的最优工程形式有待逐一验证。

因此，本文完成的工作是划出一条先验边界：在长程实践里，单次能力再强也不能替代承载结构本身。

A 受控解剖：一份长程实践的结构标本

本附录不讨论以下括号串计数问题的数学价值或竞赛意义。我们把它当作一份受控解剖材料：输入输出可机械判定，闭合标准严格（全部用例通过 + 时间/空间约束 + 非降级），且探索轨迹有完整锚点留痕。我们关心的不是解题者最终如何解出此题，而是沿着真实记录重建：哪些关键输入只能在推进中出现，哪些局部正确无法自然累积为全局闭合，以及任务最终依赖什么结构才得以跨过连续断裂。

A.1 原题陈述

以下为本题的完整原始描述。括号串 DP 是一道知名的组合优化题目，但在本附录中我们不关心其竞赛背景——只把它作为一份信息结构可被精确重建的受控标本。

时间限制：10 秒
内存限制：1024 MB

题目描述

Alice 和 Bob 喜欢研究括号串。

称长度为 $2n$ 、仅由字符 (与) 构成的括号串 $s = s_1s_2\cdots s_{2n}$ 为**合法**，当且仅当：

- 对任意前缀，左括号数量不少于右括号数量；
- 整串中左括号与右括号各有 n 个。

对一个长度为 $2n$ 的合法括号串 s ，定义 $\text{alice}(s)$ 为 s 中等于字符串 "()" 的子序列的个数。换句话说， $\text{alice}(s)$ 等于满足 $1 \leq i < j \leq 2n$ 、且 $s_i = ($ 、 $s_j =)$ 的二元组 (i, j) 的数量。

对于给定整数 a ，Alice 定义 $\text{SA}(s) = a^{\text{alice}(s)}$ 。

Bob 为同一个串 s 设计了另一种计数方式。设两个变量初值为 $l = 0$ 、 $\text{bob}(s) = 0$ 。当 $l < n$ 时依次重复以下过程：

- 令 x 为最大的正整数，使得第 $2 \times l + x$ 个前缀中，左括号 (的个数恰好等于 $l + x$ ；
- 更新 $l \leftarrow l + x$ ；
- 更新 $\text{bob}(s) \leftarrow \text{bob}(s) + (n - l)$ （注意执行顺序，要先更新 l ）。

可以证明该过程一定能进行下去，并最终得到 $l = n$ ，且每一步都能保证 $x \geq 1$ 。

对于给定常数 b ，Bob 定义 $\text{SB}(s) = b^{\text{bob}(s)}$ 。

现在给出整数 n ，对每个整数 i ($1 \leq i \leq n$)，设 $\mathcal{D}_i$ 为所有长度 $2i$ 的合法括号串集合，要求计算：

$$F_i = \sum_{s \in \mathcal{D}_i} \text{SA}(s) \times \text{SB}(s) = \sum_{s \in \mathcal{D}_i} a^{\text{alice}(s)} \times b^{\text{bob}(s)}.$$

由于答案可能很大，只需要输出 $F_i \bmod 998\,244\,353$ 即可。

输入描述

在一行上输入三个整数 n, a, b ($1 \leq n \leq 5000$; $1 \leq a, b < 998\,244\,353$)。

输出描述

在一行上输出 n 个整数，第 i 个整数表示 $F_i \bmod 998\,244\,353$ 的值。

样例

输入 #1	输入 #2
3 1 1	4 2 3
输出 #1	输出 #2
1 2 5	2 40 4544 2624512

正确解法路线。该题的最终解法经历了以下关键阶段：

- 在穷举了多条候选路线（A~K）后，通过 $n = 8$ 的全量数值验证，确

41

- 认该问题等价于组合数学中已知的 (q, t) -Catalan 数^[9-10];
2. 利用 Garsia-Haglund 递推公式^[11] 将问题转化为 $O(N^3/6)$ 的动态规划;
 3. 在判题机时限压力下, 通过 q -阶乘分离与 NTT (数论变换) 卷积将复杂度降至 $O(N^2)$;
 4. 最终处理模逆元存在性等边界条件后通过全部测试用例。

这个解法的每一步推进, 都依赖于上一步执行后产生的新信息——等价性无法靠”思考”得出、复杂度无法靠”分析”预判、边界条件无法在起点获知。它天然是一份长程实践的结构标本。

A.2 标本选择：为什么这道题适合解剖

第一, 题目是封闭形式系统。给定 $n \leq 5000$ 、模数 998244353、时限 10 秒、内存限制 1 GB, 计算所有长度为 $2i$ 的合法括号串上 $a^{alice(s)}b^{bob(s)}$ 的总和——输入输出完全形式化, 不存在自然语言歧义。这排除了”现实环境太乱”的借口: 任何失败都不能归因于需求理解偏差或规范漂移。

第二, 闭合标准完全可机械判定。判题系统返回单一的 AC/WA/TLE 判决, 不存在”方向对了””大体正确”等模糊中间态。21 个测试用例全部通过、时间 $\leq 10s$ 、内存 $\leq 1\text{ GB}$ 、单文件实现、不使用非标准降级策略——这五个条件共同构成一个可自动执行的闭合判据。排除了”评价标准模糊”的借口。

第三, 本题的递推规模直接触发了 §3 定义的媒介强制。Catalan 数的组合爆炸与 $O(N^2)$ DP ($N = 5000$) 产生了一个远超任何人类工作记忆或 LLM 上下文窗口的状态空间——每一步推进所依赖的等价性结论、已证伪方向、边界条件判据, 在下一步开始时已无法被稳定装载进当前信息集。这意味着: 不是”记录得好所以能回放”, 而是”不记录就无法推进”。任务中产生的结构记录——spec 文件中的已证真/已证伪表、作废索引、禁止重试路线清单、判题反馈历史、GATE 闭合表——不是工程便利, 而是对媒介强制的结构补偿。排除了”只是模型不够强或记性不好”的替代解释: 长程性来自信息结构的客观约束, 而非执行者的认知局限。

第四, 这道题中外部漂移几乎不存在。题目约束、测试数据、判题机行为在任务期间保持稳定。正因为如此, 如果它仍然表现出显著的长程性, 就说明长程性不是现实噪声的副产品, 而是任务内部信息结构的结果——关

键输入沿时间轴逐步生成这一事实，即使在没有外部世界变化时也成立。

数据可用性说明。该标本的完整原始研究资料——包括全部检查点执行轨迹、证伪历史、锚点文件（spec.md、current-note.md 等）、判题反馈记录——已开源至 https://github.com/jopsammy/task1_raw_repo，供读者对照本文分析自行检验。

A.3 检查点重建：认知推进的断面

以下按长程实践的结构框架重建任务轨迹。表中每个条目对应一个信息生成阶段，每阶段回答三个问题：当前已有信息是什么、下一步还缺什么关键输入、这个关键输入为什么此刻不在当前信息集中。

Table 1: 检查点序列 (C1–C3): 信息增长与认知缺口的逐点显形

编号	当前已有信息	缺失的关键输入	为什么不可得
C1	题目约束 ($N \leq 5000$, 10s, 1GB, MOD= 998244353)、alice 沿标准 Catalan 分解可处理、bob 的逐前缀批次扫描定义	哪种分解范式能同时兼容 alice 的树结构和 bob 的 x 序列结构	这个真值不是靠思考就能确定的——必须实际写出 DP、跑穷举验证、对照真实 wrapping 数据后才能固定。在未执行这些动作之前, 世界尚未生成这一信息
C2	多条候选路线 (A~K) 已被探索并记录, 部分已被局部验证	这些路线在真实 wrapping 下是否全局闭合	wrapping 等价于完整高度函数, 其行为不能被任何低维 DP 状态无损预测。换句话说, 不存在一个比”逐路线执行验证脚本”更便宜的方法来获取这一真值——必须逐路线执行
C3	大量路线已被证伪并封禁	问题是否等价于已知组合对象 ((q, t) -Catalan)	等价性不是靠”看出”的——需要将 alice/bob 映射到 Dyck 路径的 area/bounce 统计量, 再对 $n = 8$ (共 1430 个 Dyck 词) 做全量数值验证。在全量验证完成之前, 等价性只是一个假说, 不是一个事实

Table 2: 检查点序列（C4–C7，续）

编号	当前已有信息	缺失的关键输入	为什么不可得
C4	确认等价于 (q, t) -Catalan, Garsia-Haglund 递推可用 ^[11] , 复杂度 $O(N^3/6)$	$O(N^3/6)$ 能否在 10s 内通过判题机	判题机微架构（CPU 型号、缓存层级、编译器优化策略）未知，本地 8.89s 不代表判题机也能通过。这个真值在递交判题之前不存在——必须实际触发判决
C5	v11 判题 17/21, 4 例超时；208 亿次点积确认为代数下限	是否存在不改变循环顺序而从代数层面降阶的路径：q-阶乘分离 + NTT 卷积	q-阶乘分离是否成立，必须先完成形式化推导和 $n \leq 8$ 全量验证后才能确定。在推导完成之前，这个信息尚不存在——不是“已知但不方便算”
C6	v14 NTT 路径 $n \leq 8$ 全量通过， $N = 5000$ 用时 7.66s	对所有可能的 a 是否都存在模逆元	只有当某个 a 的乘法阶 $\leq N + 2$ 时 q-阶乘才会出现零因子。在预检测完成之前，这个真值不可能被预知——必须通过计算才能被世界固定
C7	v15 判题 19/21, 2 例 WA	错误来自算法设计还是实现细节（整数溢出、模运算边界）	必须定位到具体用例、分析对应 a 的乘法阶后才能确定错误类别。在定位完成之前，错误的类别自身就是一个未定的信息

表 1 和表 2 列出了认知缺口最显著的七个信息生成阶段。每个阶段内包含大量子级断面（已证伪路线的逐条淘汰、判题反馈的逐次响应等），但“关键输入在被执行动作触发前不作为确定事实存在”这一结构特征在七个阶段中一致重演。这正是正文 §3 中认知缺口的纯粹形态。

A.4 断裂定位：四种来源在封闭系统中的具体显现

以下将正文 §3 的四类来源映射到标本的真实轨迹上。此处不做定义重复，只提供结构切片。

认知缺口（标本中最强的来源）。三个决定性真值均在执行动作前不存在：(1) wrapping 是否可由局部参数确定——这个真值在执行 $n = 4$ （共 14 个 Dyck 词）和 $n = 8$ （共 1430 个 Dyck 词）的穷举对照之前不存在；(2) Garsia-Haglund 递推是否等价于原问题——这个真值在 $n \leq 8$ 全量数值验证完成之前不存在；(3) 判题机是否能在 10s 内跑完 $O(N^3/6)$ ——这个真值在递交判题之前不存在。这三个真值都不是“藏起来等待被发现”的事实，而是“必须通过执行特定动作才会被世界生成”的事实。这正是认知缺口的定义性特征：真值本身沿时间轴逐步生成，而不是沿空间轴逐步被发现。

决策分叉（标本中次强的来源）。路线 A 至 L 的连续分叉构成了典型的决策分叉链：基于树分解的 wrapping 局部化方案（路线 A~D）被证伪后，转向基于 x 序列的批次合并方案（路线 E~H），再转向 $L(i)$ 序列表示（路线 I~K）。每条被排除的路线都消耗了真实的探索与验证成本（穷举脚本运行时间、形式化推导时间），而排除结论本身也必须被稳定保存为后续搜索空间的永久边界——即“禁止重试路线”清单。未被选中的路径不是“没有被尝试”，而是“被尝试后被证伪”，且其证伪结论是后续所有步骤的共同前提。

媒介强制（标本中较弱但真实存在）。在 102 小时的跨日推进中，全部失败历史、作废结论、例外条件、当前最优版本和未闭合项清单不可能稳定压在单轮上下文或单次会话中。spec.md 和 current-note.md 的外置锚点体系——以三段交接结构（工程过程 / 交接状态 / 最终结果）维持跨日状态——正是媒介强制的工程回应。媒介强制在此不是最戏剧化的断裂来源，但它是连续性得以维持的前提：没有外置锚点，跨日推进就会退化为每次从零重来，而与上一日的推理积累完全失联。

外部漂移（标本中几乎不存在——这是优点）。本题是封闭形式系统，外部世界状态在任务期间基本不变。任务约束、测试数据、判题机行为均保持稳定。标本对“外部漂移”的覆盖很弱，但这恰恰是它作为受控标本的价值：在排除了外部漂移之后，任务仍然在 $C1 \rightarrow C7$ 的整个轨迹上表现出显著的长程性。这说明长程性不来自“世界会变”，而来自“关键信息沿时间轴逐

步生成”这一更根本的结构事实。

A.5 连续性修复：承载结构如何自然长出

以下展示标本推进过程中实际依赖的结构对象，并将它们映射到正文 §4 的三层下限。这里的核心主张是：这些对象不是解题者预先设计的制度，而是在断裂压力下被迫长出的结构。

Table 3: 结构对象 → 正文条件 → 被迫长出的机制

任务中实际存在的结构对象	对应正文条件	如何在标本中被迫长出
GATE 表 (21/21 AC + 时间 + 内存 + 单文件 + 非降级)	目的闭合	不是设计偏好——没有它, 17/21 已过就只是模糊进度感而非可操作闭合判据, 执行者无法确定”什么时候算完成”
”禁止重试路线”清单	价值闭合	不是管理要求——没有它, 后续执行者会反复坠入已被证伪的旧路线 (路线 A~K 中的已封禁项), 每一次重试都消耗真实的探索成本
从组合分解到代数降阶的多阶段推进	检查点分解	不是预先规划——是在每条旧范式 (树分解、 x 序列、 $L(i)$ 表示) 被证伪后才逐步形成的阶段边界。检查点的划分边界由断裂位置决定, 而非由事前的路线图决定
spec.md 与 current-note.md 之间的双向同步	状态传递	不是文档规范——没有它, 跨日恢复时前一日的判断依据和封禁清单在断面处蒸发, 新会话不知道哪些路线已排除、哪个版本是当前最优、哪些反例已被发现
已证真 / 已证伪 / 作废的三态标记	未闭合标记	不是分类癖好——没有它, 已被推翻的结论 (如“(A)B 分解对 bob 成立”的早期假说) 会在后续步骤中被误当作前提继续使用, 导致建立在错误基础上的推导浪费
作废索引 (被推翻结论的永久留痕)	顺序约束 + 状态传递	不是存档洁癖——没有它, 连续分叉后的知识边界无法维持。被放弃的路线需要被”记住为什么被放弃”, 否则系统会在旧坑和新坑之间反复震荡

表 3 的六行映射共同揭示了一个模式：每一个结构对象都不是因为”应该这样做”而被设计出来的，而是因为”不这样做任务就会在断裂带上失稳”而被任务本身要求出来的。这些对象是长程实践的承载骨架，不是锦上添花的最佳实践。

A.6 为什么裸连模型会稳定失手

以下论证不提供“模型够不够强”的结论，而是沿前四段的证据链让结构原因自行显现。

裸连不缺局部推理能力。标本中大量局部步骤——Catalan 结构的识别、 (q, t) -Catalan 的等价性判断、 q -阶乘分离的代数推导、NTT 卷积的设计——在单次推理层面都是真正的数学洞察。在 $C1 \rightarrow C3$ 的早期阶段，单次推理能力足以完成穷举验证、等价性确认和初始递推设计。

裸连缺的是“使局部正确可累积”的连续性对象。以下六类对象是局部正确转化为全局闭合的必要承载——它们默认不在单轮对话中被稳定持有：

1. 可回放的证伪历史：知道哪些路线已被排除、为什么被排除；
2. 稳定的禁止重试边界：已被证伪的路线不会在新会话中被重新尝试；
3. 当前唯一真相源（`spec.md`）：所有执行者共享同一份“当前已知为真”的清单；
4. 中断后可续接的断面状态（`current-note.md` 的三段交接）：跨日恢复时不丢失前一日判断依据；
5. 独立于执行者自述的闭合证据：判题反馈、全量验证脚本输出，不是“我觉得对了”；
6. 当新反例出现时对整条路线图的重写能力： $v14 \rightarrow v15$ 的路径升级（ q -阶乘从因子结构改为 NTT 卷积），要求执行者回到更上游的假设层重新判断，而不只是追加补丁。

这六类对象是长程实践的承载骨架。它们的共同特征是：都不在单次推理的输入-输出空间内，而属于“跨检查点维持连续性”所必需的元结构。

失手的结构原因，不是智力原因。裸连模型在标本上的失手，不是因为某一步算错了，而是因为以上六类对象在默认的单轮对话中不被稳定持有。更进一步——即使假设一个无限聪明的单次推理者存在，它也无法在单次会话中预知 `wrapping` 的全局性（ $C2$ ）、Garsia-Haglund 的等价性（ $C3$ ）、判题机的微架构（ $C4$ ）、坏 a 的零因子（ $C6$ ）——因为这些真值在被实际执行和验证之前根本就不存在于世界中。问题不位于求解器内部，而位于求解器与连续性对象之间的缺口。只要这个缺口不被单独识别和补偿，局部推理的强度就不会自动转化为跨检查点的闭合。

A.7 标本的边界：覆盖什么，不覆盖什么

强覆盖。认知缺口（C1→C7 全程）、媒介强制（Catalan 组合爆炸与 $O(N^2)$ DP 的状态空间远超上下文窗口上限，锚点体系是对此的结构补偿）、决策分叉（路线 A~L）、目的闭合（GATE 表）、检查点分解（从组合分解到代数降阶）、状态传递（spec/note 双向同步）、未闭合标记（三态标记）。这些是标本展示最充分的结构维度。

中等覆盖。价值闭合（“禁止重试路线”清单作为人类裁决的外置边界）。标本在这一维度上提供了真实但有限的证据。

弱覆盖。外部漂移（封闭题，任务期间环境基本不变）、工程补偿条件的大部分（现实噪声——人员变动、需求变更、工具链升级——在封闭系统中被大幅削减）。

这些“弱覆盖”不是标本的缺陷。恰恰因为外部漂移被排除，标本才证明了长程性的根源不在环境噪声，而在信息沿时间轴生成这一更根本的结构事实。工程补偿条件的弱覆盖则说明：封闭世界中的长程实践主要受逻辑层约束（正文 §4 的 6 条必要条件），而真实世界中的长程实践还需要补偿层的噪声缓冲（7 条补偿条件）。两者不是矛盾，而是标本与现实的合理差异——标本通过削减现实噪声，使逻辑层的结构必要性变得可见。

B 致谢

- 感谢杭州市上城区钱江新城中央商务产业社区在调研对接、基础压力测试样例与真实业务用例方面提供的关键性支持
- 感谢深度求索（DeepSeek）提供 Deepseek-v4-pro 基座支持；附录标本的 102 小时压力运行由该基座在锚点契约式人机协同工程范式（AC 范式 V6）下完成
- 感谢杭州市协同引荐平台社区（SRO）提供实质性用例支持
- 感谢彗星计划（Comet Project）成员提供技术支持
- 感谢 AI-Zion 社区大量成员提供了富有启发性的讨论，给出宝贵的建议

- 感谢 TRAE 提供关键 IDE 环境技术支持
- 感谢 Cherry Studio 提供关键技术支持

References

- [1] LIU F, LV B, LIU Y. From Observer to Agent: On the Unification of Physics and Intelligence Science[Z/OL]. 2024. <https://www.preprints.org/manuscript/202410.0479/v1>.
- [2] LIU N F, LIN K, HEWITT J, et al. Lost in the Middle: How Language Models Use Long Contexts[J/OL]. Transactions of the Association for Computational Linguistics, 2023, 11: 157-173. <https://arxiv.org/abs/2307.03172>. DOI: 10.1162/tac1_a_00638.
- [3] SIMON H A. A Behavioral Model of Rational Choice[J/OL]. Quarterly Journal of Economics, 1955, 69(1): 99-118. DOI: 10.2307/1884852.
- [4] Anthropic. Best Practices for Claude Code[EB/OL]. 2025 [2026-05-21]. <https://www.anthropic.com/engineering/claude-code-best-practices>.
- [5] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: Can Language Models Resolve Real-world Github Issues?[C/OL]//The Twelfth International Conference on Learning Representations. 2024. <https://openreview.net/forum?id=VTF8yNQM66>.
- [6] CHOWDHURY N, AUNG J, SHERN C J, et al. Introducing SWE-bench Verified[EB/OL]. 2024 [2026-05-21]. <https://openai.com/index/introducing-swe-bench-verified/>.
- [7] AMERSHI S, WELD D, VORVOREANU M, et al. Guidelines for Human-AI Interaction[C/OL]//CHI 2019. 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>.
- [8] HORVITZ E. Uncertainty, Action, and Interaction: In Pursuit of Mixed-Initiative Computing[J/OL]. Intelligent Systems, 1999: 17-20. <https://www>

[.microsoft.com/en-us/research/publication/uncertainty-action-interaction-pursuit-mixed-initiative-computing/](https://www.microsoft.com/en-us/research/publication/uncertainty-action-interaction-pursuit-mixed-initiative-computing/).

- [9] GARSIA A M, HAIMAN M. A Remarkable q, t -Catalan Sequence and q -Lagrange Inversion[J/OL]. Journal of Algebraic Combinatorics, 1996, 5(3): 191-244. DOI: [10.1023/A:1022476211638](https://doi.org/10.1023/A:1022476211638).
- [10] HAGLUND J. The q, t -Catalan Numbers and the Space of Diagonal Harmonics: vol. 41[M]. American Mathematical Society, 2008.
- [11] GARSIA A M, HAGLUND J. A Proof of the q, t -Catalan Positivity Conjecture[J/OL]. Discrete Mathematics, 2002, 256(3): 677-717. DOI: [10.1016/S0012-365X\(02\)00343-6](https://doi.org/10.1016/S0012-365X(02)00343-6).